

Neural Network

Model

Model

An informative representation of an object, person or System

Model

An informative representation of an object, person or System

- Physical Model

Model

An informative representation of an object, person or System

- Physical Model
- Conceptual Model

Mathematical Model

Mathematical Model

Nothing but a mathematical function

Mathematical Model

Nothing but a mathematical function

- Input variable

Mathematical Model

Nothing but a mathematical function

- Input variable
- Parameter

Mathematical Model

Nothing but a mathematical function

- Input variable
- Parameter
- Output

Mathematical Model

Nothing but a mathematical function

- Input variable
- Parameter
- Output

$$y = f(x|w)$$

Mathematical Model

Nothing but a mathematical function

- Input variable
- Parameter
- Output

$$y = f(x|w) \longrightarrow \textit{Abstract}$$

Mathematical Model

Nothing but a mathematical function

- Input variable
- Parameter
- Output

$$y = f(x|w) \longrightarrow \textit{Abstract}$$
$$y = mx + c$$

Mathematical Model

Nothing but a mathematical function

- Input variable
- Parameter
- Output

$$y = f(x|w) \longrightarrow \textit{Abstract}$$

$$y = mx + c \longrightarrow \textit{Concrete}$$

Helping a cousin with MBA admission

Helping a cousin with MBA admission

CGPA

CAT Score

Helping a cousin with MBA admission

Helping a cousin with MBA admission

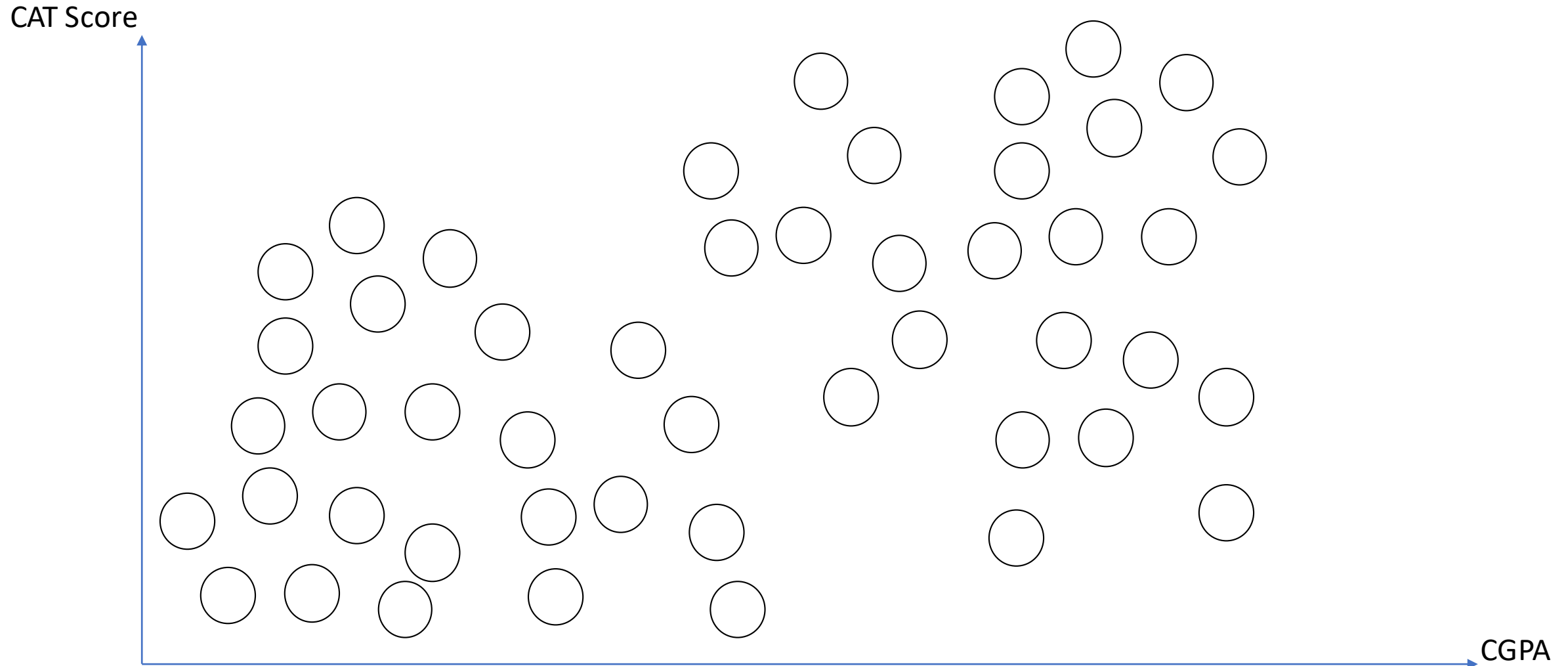
CAT Score



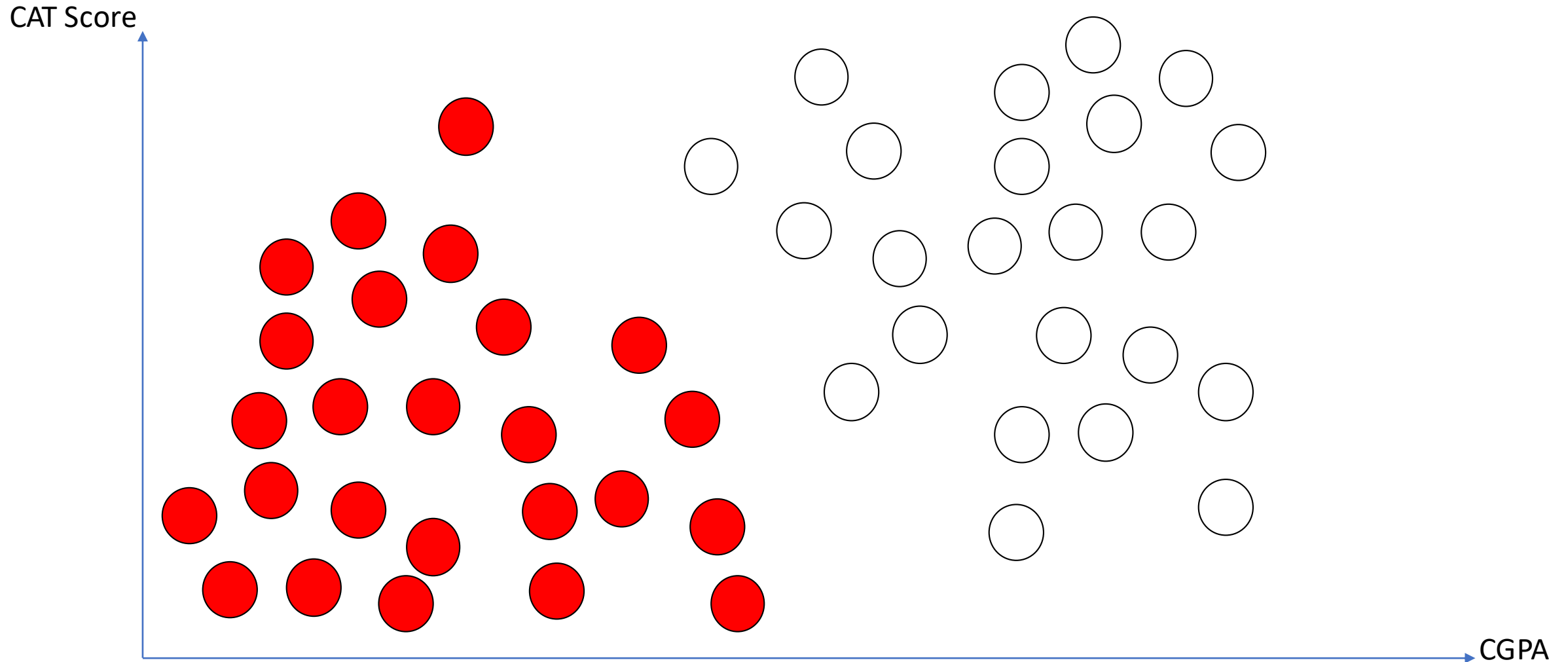
CGPA



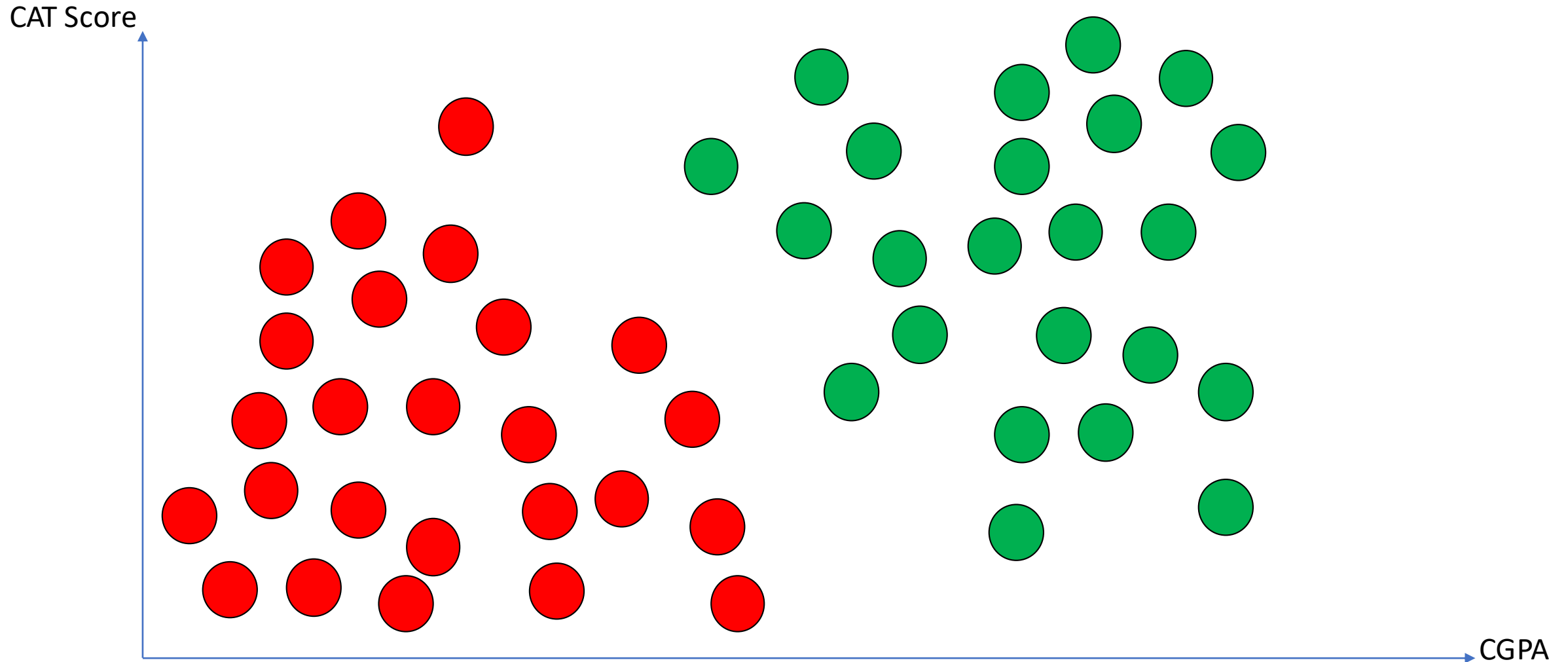
Helping a cousin with MBA admission



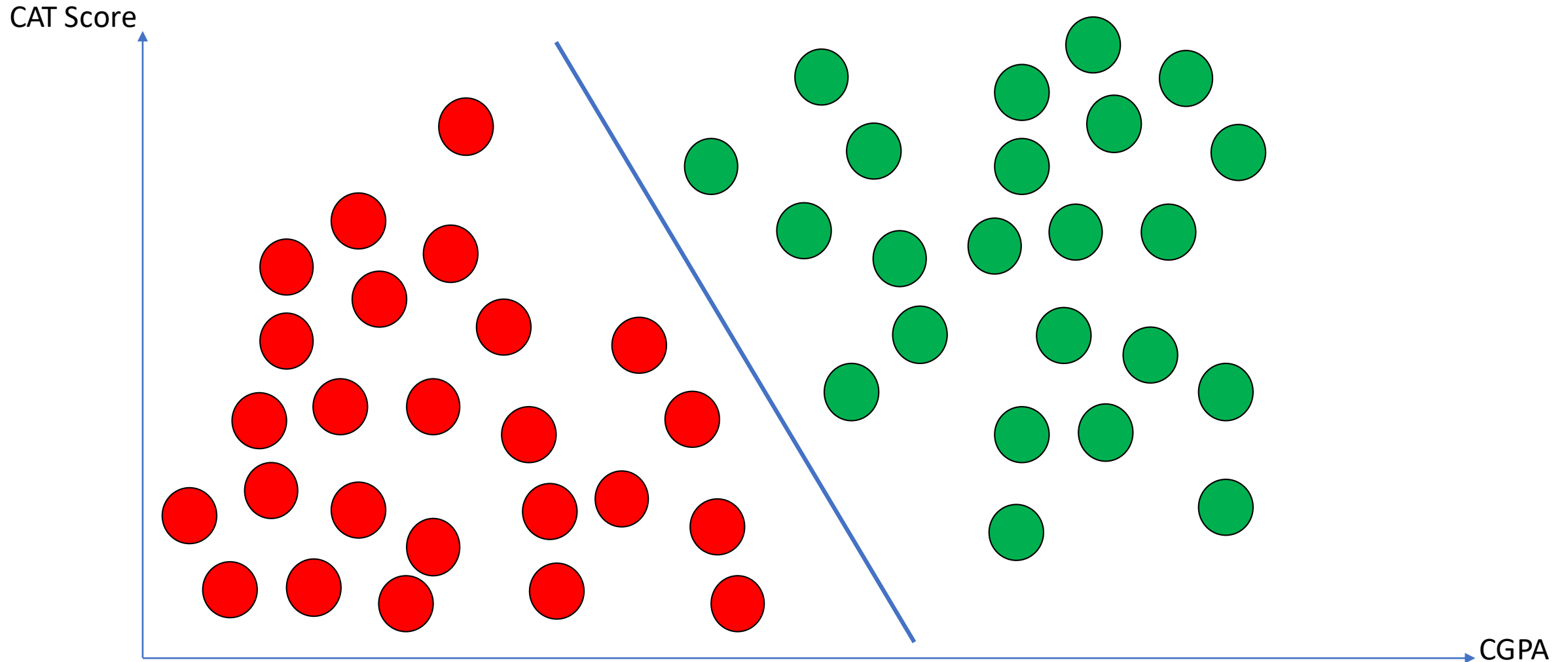
Helping a cousin with MBA admission



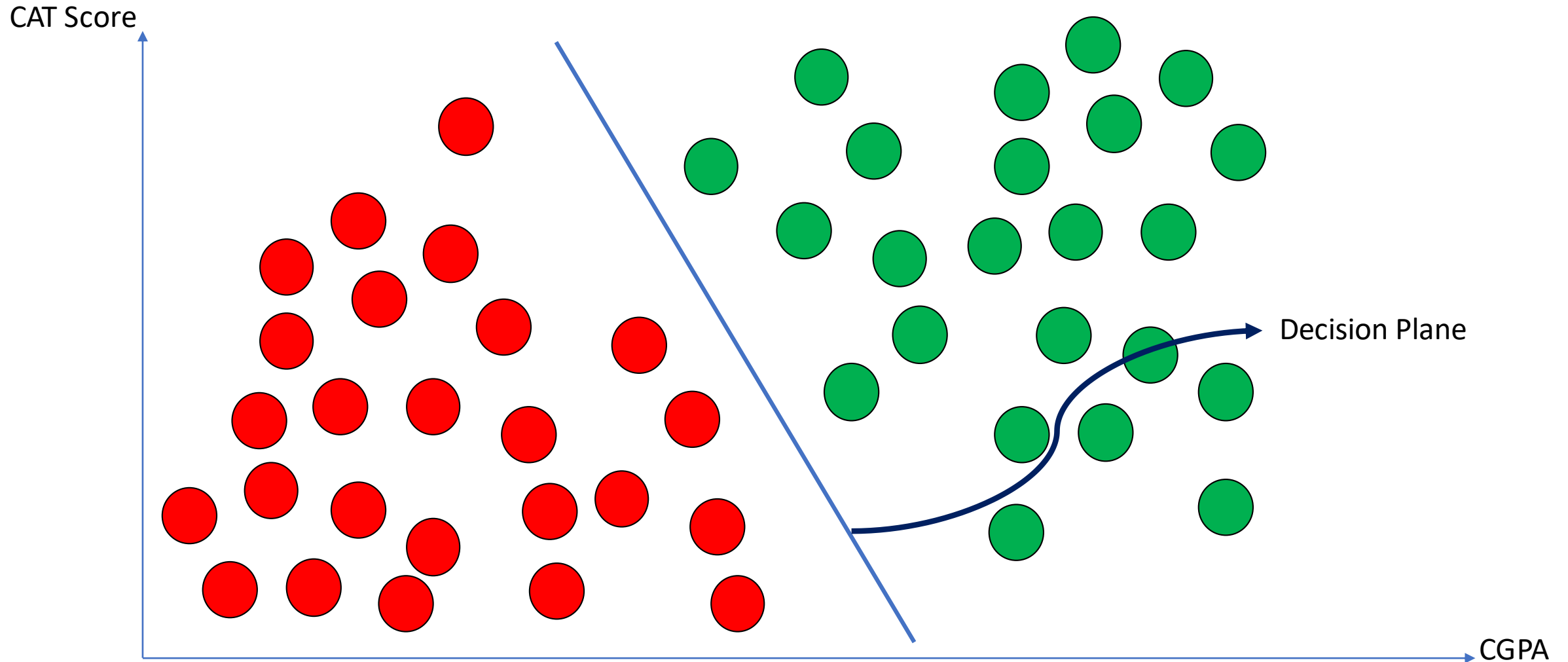
Helping a cousin with MBA admission



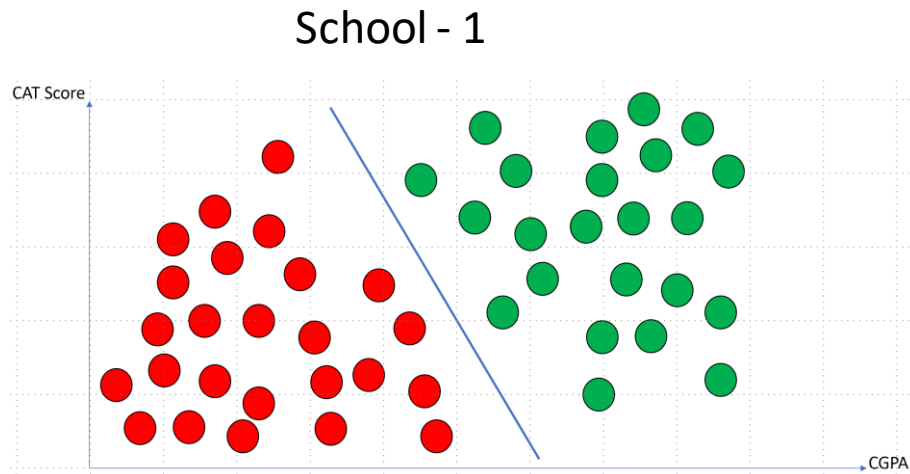
Helping a cousin with MBA admission



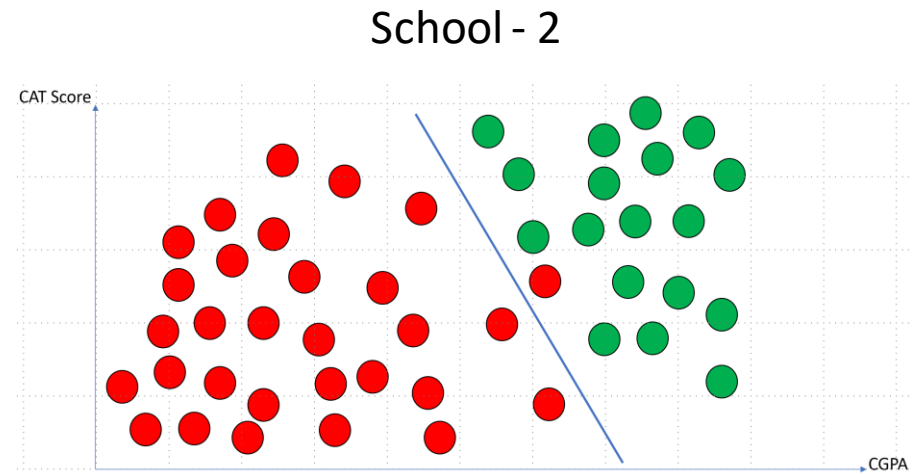
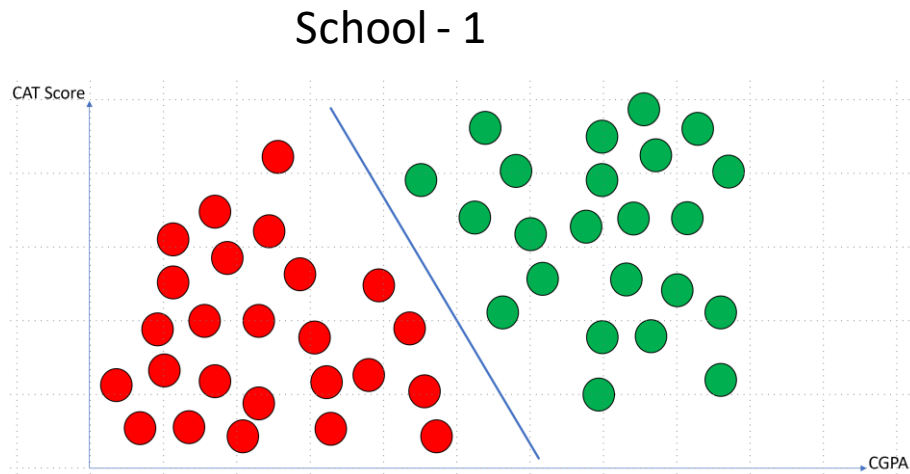
Helping a cousin with MBA admission



Helping a cousin with MBA admission

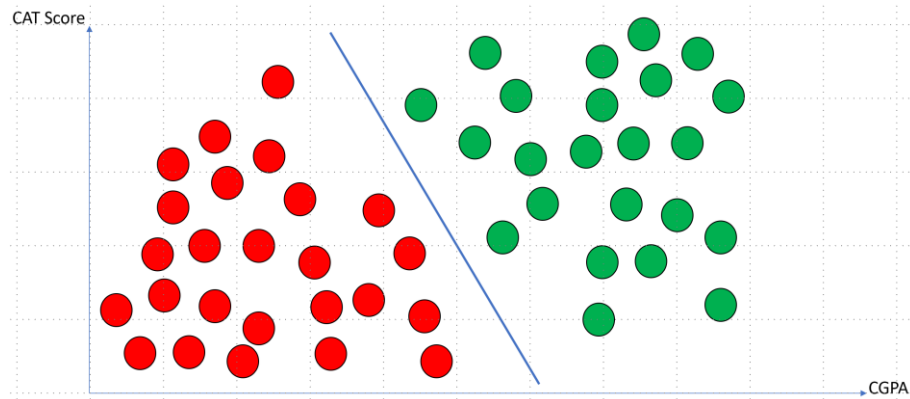


Helping a cousin with MBA admission

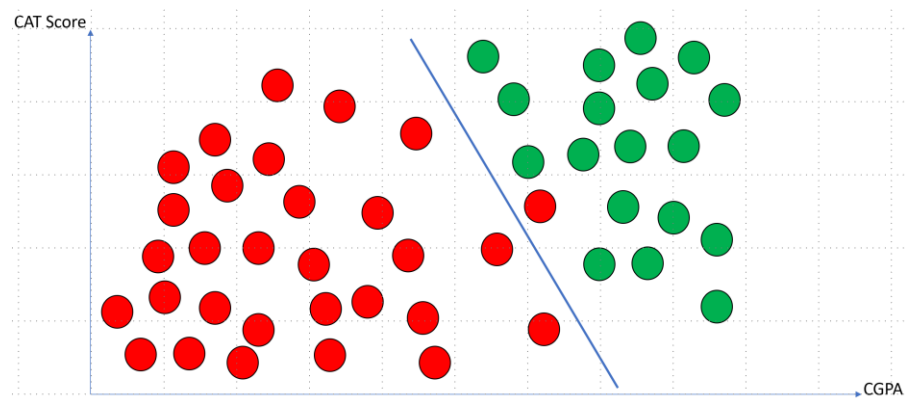


Helping a cousin with MBA admission

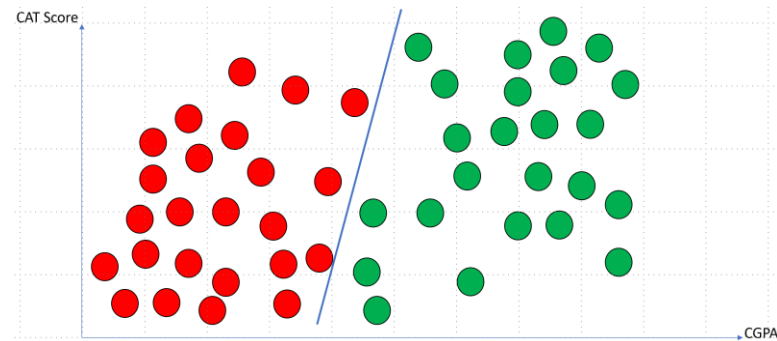
School - 1



School - 2



School - 3



What's the problem?

What's the problem?

- Have to manually plot and tell the answer

What's the problem?

- Have to manually plot and tell the answer
- May not be able to handle multiple requests

Solution?

Solution?

- Automation

Solution?

- Automation
- Need to tell the computer about the decision plane and let the computer decide

Solution?

- Automation
- Need to tell the computer about the decision plane and let the computer decide
- The decision plane is nothing but a mathematical equation

Solution?

- Automation
- Need to tell the computer about the decision plane and let the computer decide
- The decision plane is nothing but a mathematical equation
- Made of **Input variables** and **Parameters**

Solution?

- Automation
- Need to tell the computer about the decision plane and let the computer decide
- The decision plane is nothing but a mathematical equation
- Made of **Input variables** and **Parameters**
- We know the values of **input variables**

Solution?

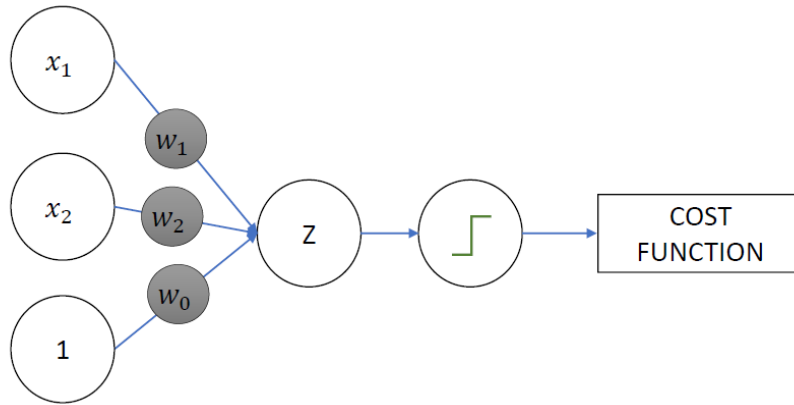
- Automation
- Need to tell the computer about the decision plane and let the computer decide
- The decision plane is nothing but a mathematical equation
- Made of **Input variables** and **Parameters**
- We know the values of **input variables**
- Unknown values are **Parameters**

Solution?

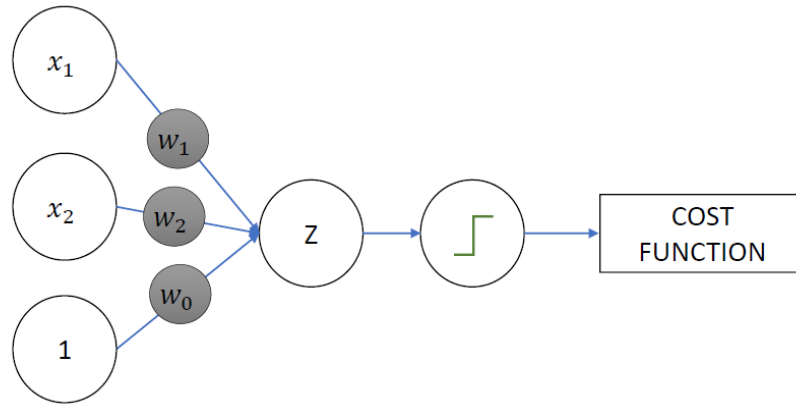
- Automation
- Need to tell the computer about the decision plane and let the computer decide
- The decision plane is nothing but a mathematical equation
- Made of **Input variables** and **Parameters**
- We know the values of **input variables**
- Unknown values are **Parameters**
- ML Based Classification model

ML Based Classification Model

ML Based Classification Model

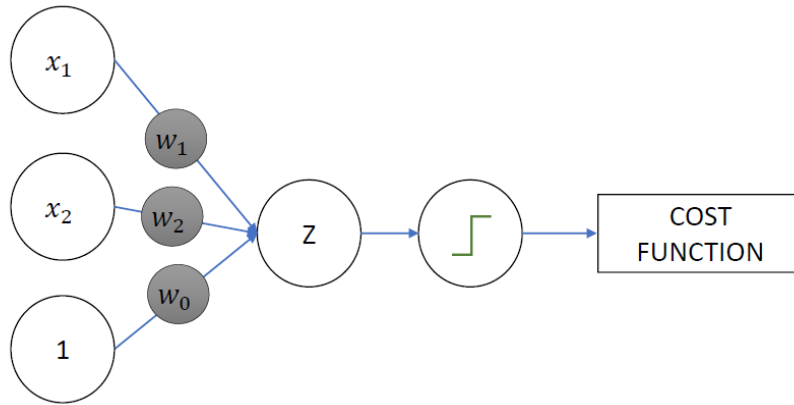


ML Based Classification Model



$$z = w_0 + w_1x_1 + w_2x_2 \text{ where } z \in (-\infty, +\infty)$$

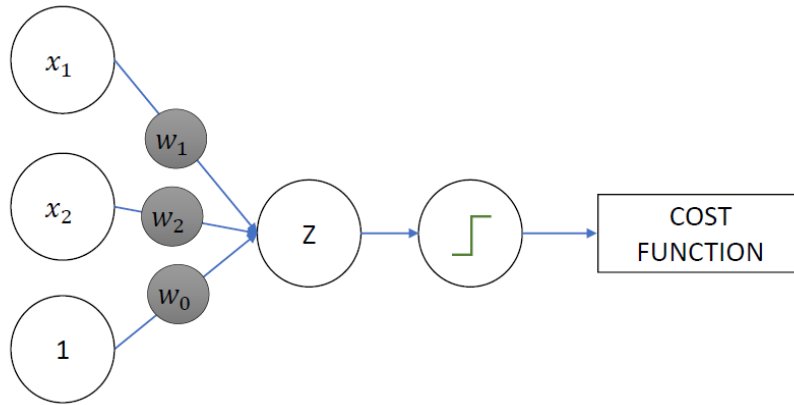
ML Based Classification Model



$$z = w_0 + w_1x_1 + w_2x_2 \text{ where } z \in (-\infty, +\infty)$$

$$y = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

ML Based Classification Model

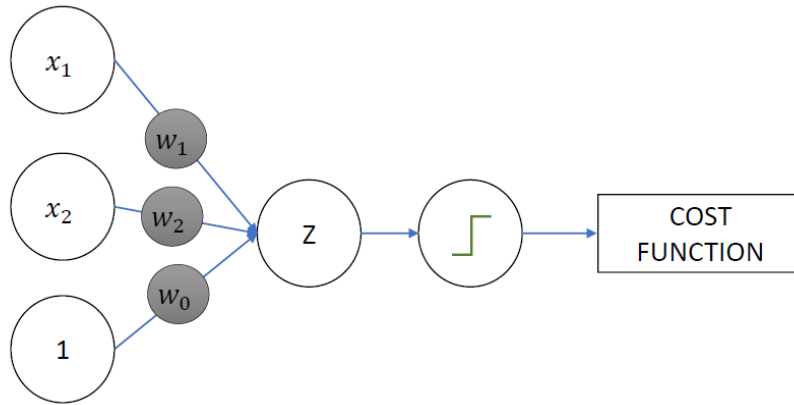


$$z = w_0 + w_1x_1 + w_2x_2 \text{ where } z \in (-\infty, +\infty)$$

$$y = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

w_0	w_1	w_2
-18	5	1

ML Based Classification Model



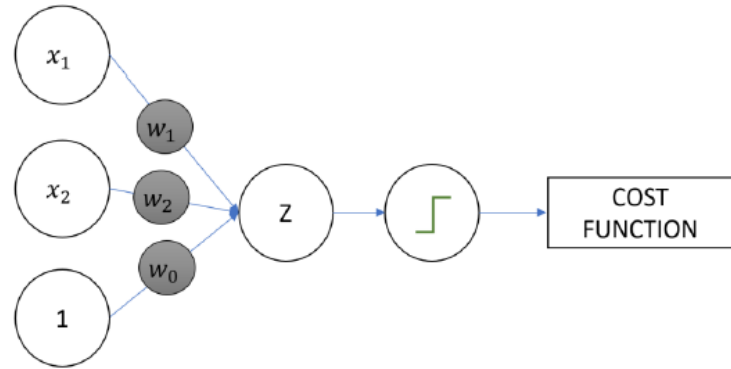
$$z = w_0 + w_1x_1 + w_2x_2 \text{ where } z \in (-\infty, +\infty)$$

$$y = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

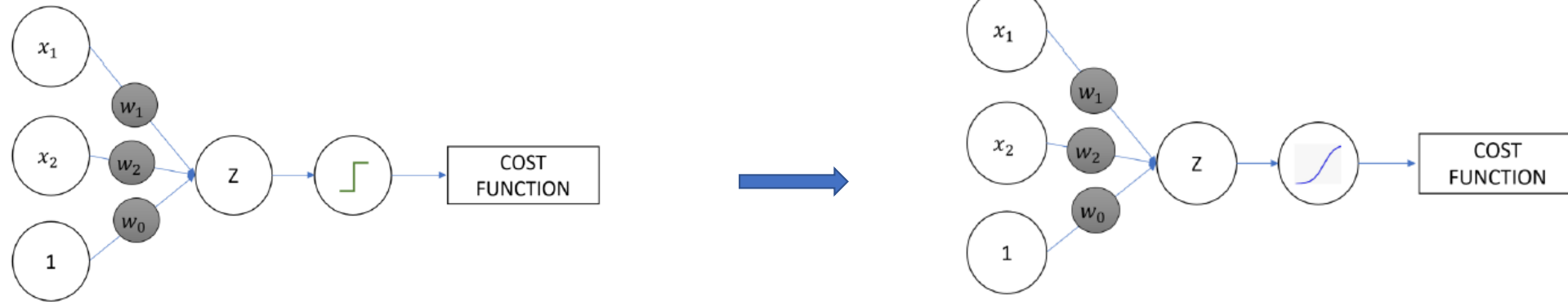
w_0	w_1	w_2
-18	5	1

```
def admission(x1, x2):  
    z = -18+5*x1+x2  
    if z>0:  
        return 'Admitted'  
    else:  
        return 'Not Admitted'
```

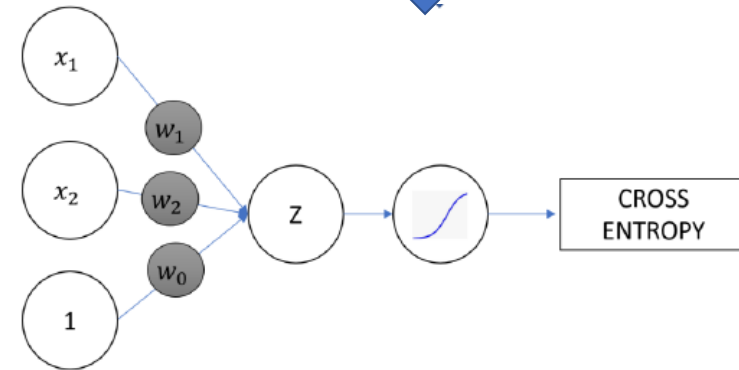
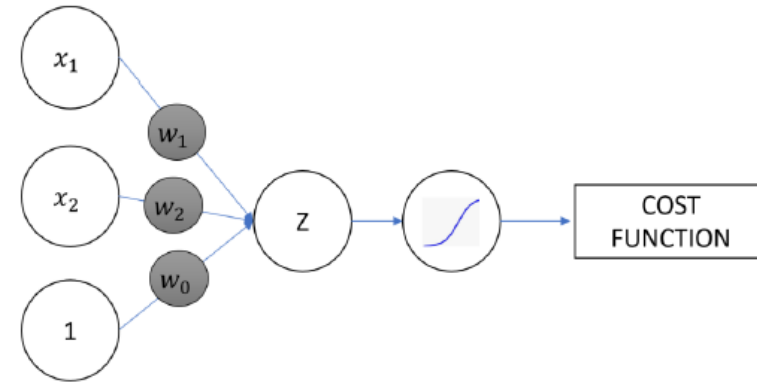
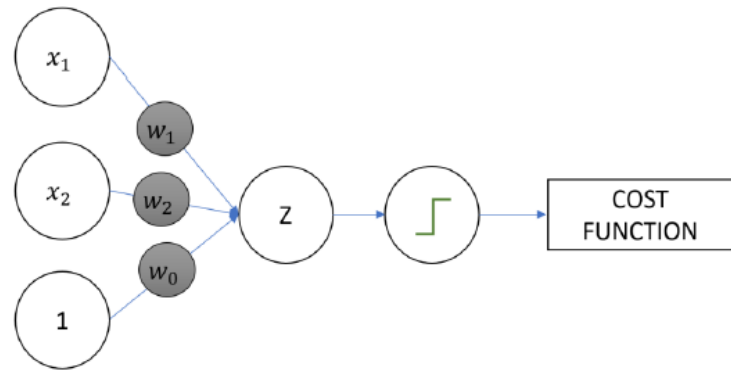
ML Based Classification Model



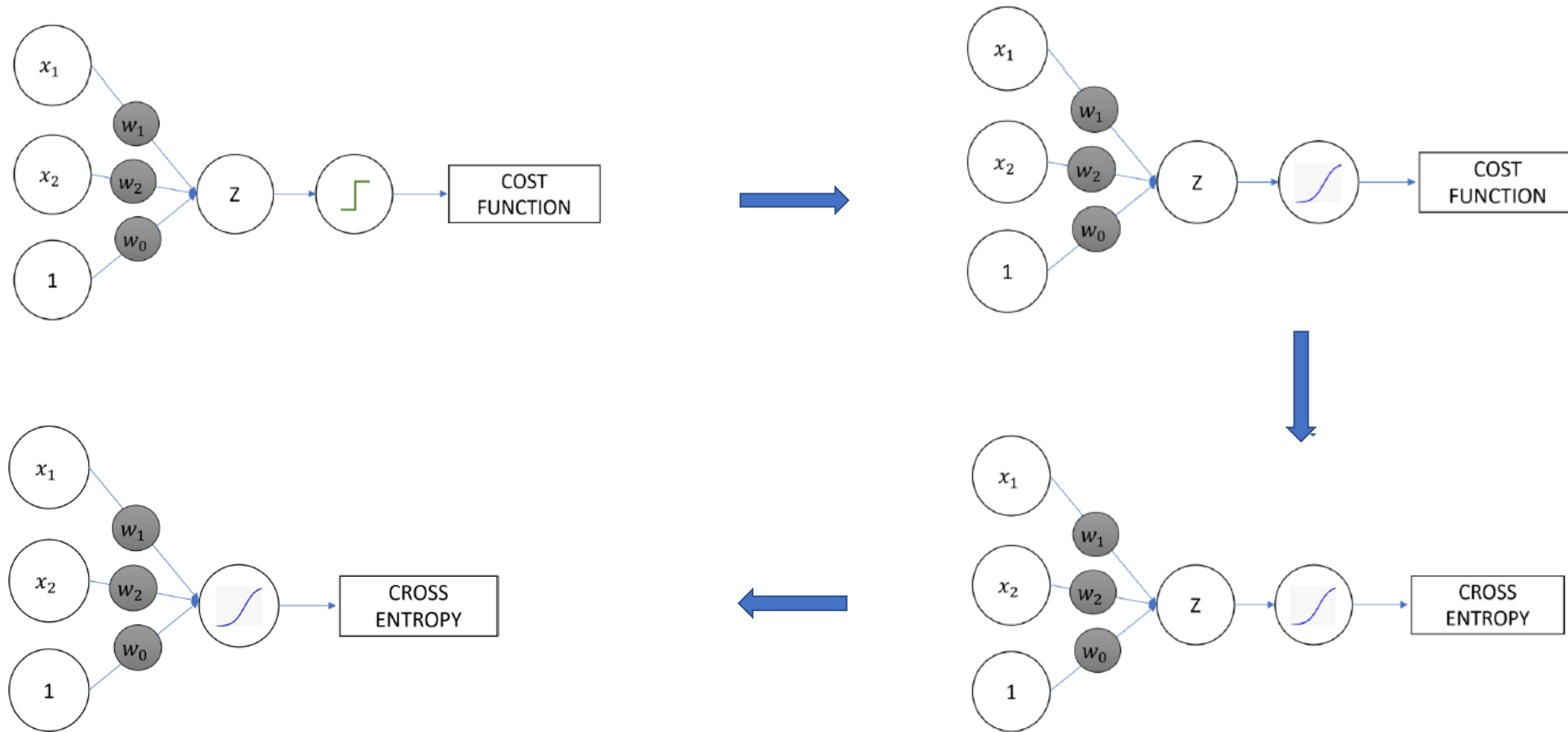
ML Based Classification Model



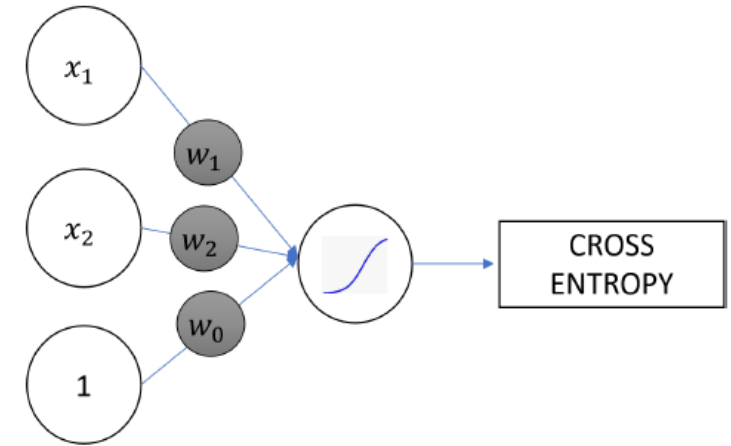
ML Based Classification Model



ML Based Classification Model



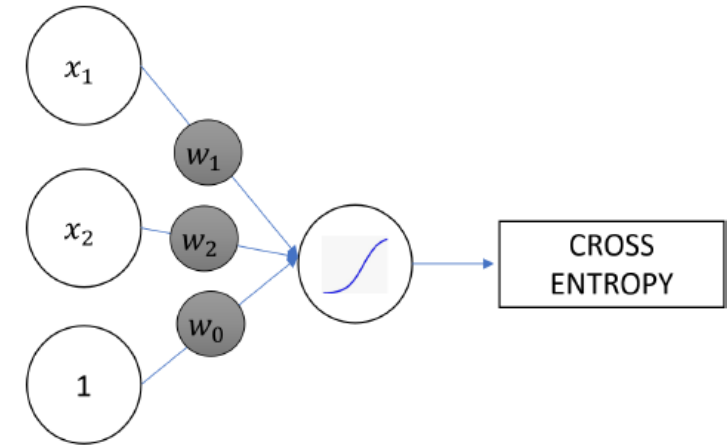
ML Based Classification Model



ML Based Classification Model

$$\text{Sigmoid} = \frac{1}{1+e^{-z}}$$

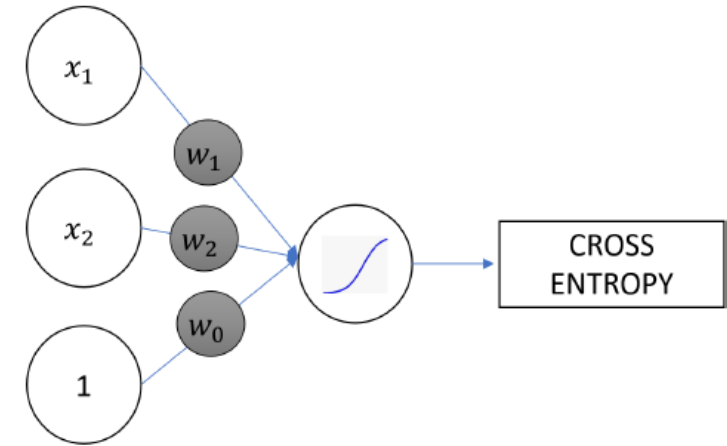
$$P(y = 1) = \hat{y} = \frac{1}{1+e^{-z}}$$



ML Based Classification Model

$$\text{Sigmoid} = \frac{1}{1+e^{-z}} \quad P(y = 1) = \hat{y} = \frac{1}{1+e^{-z}}$$

$$\text{Binary Cross Entropy}(\mathcal{L}) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$$

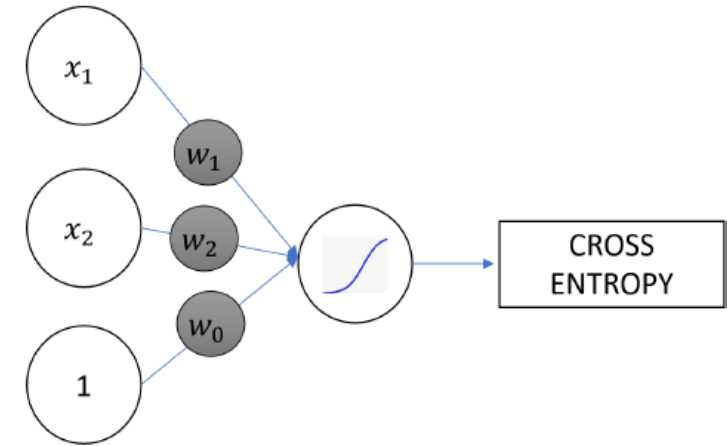


ML Based Classification Model

$$\text{Sigmoid} = \frac{1}{1+e^{-z}} \quad P(y = 1) = \hat{y} = \frac{1}{1+e^{-z}}$$

$$\text{Binary Cross Entropy}(\mathcal{L}) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$$

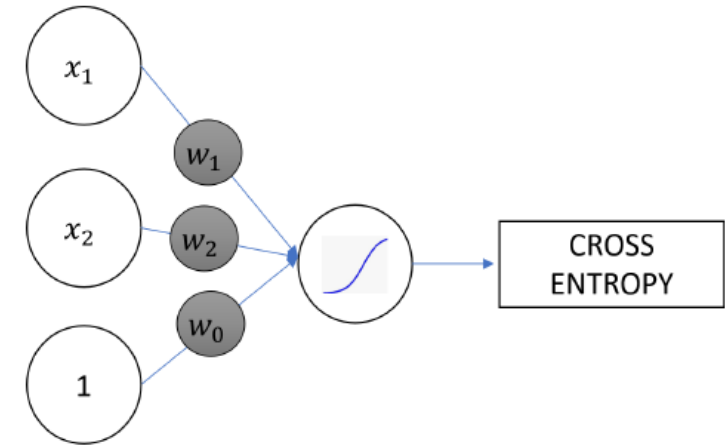
Get optimal values for W_1, W_2, W_0 by minimizing $\mathcal{L}$



ML Based Classification Model

$$\text{Sigmoid} = \frac{1}{1+e^{-z}} \quad P(y = 1) = \hat{y} = \frac{1}{1+e^{-z}}$$

$$\text{Binary Cross Entropy}(\mathcal{L}) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$$



Get optimal values for W_1, W_2, W_0 by minimizing $\mathcal{L}$

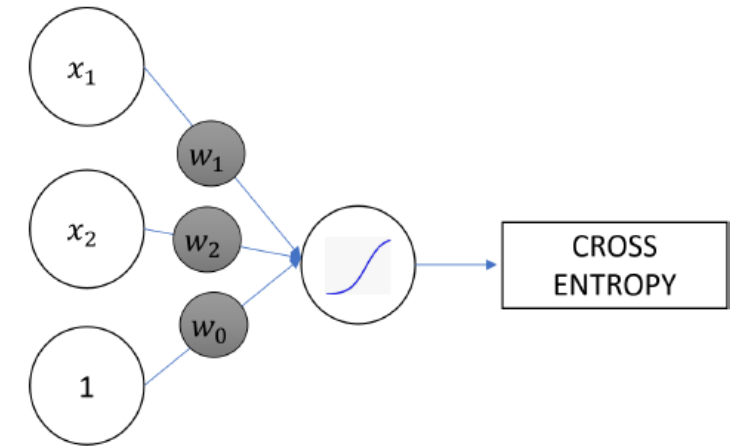
Gradient Descent

$$w_1 := w_1 - \alpha \frac{\partial \mathcal{L}}{\partial w_1} \quad w_2 := w_2 - \alpha \frac{\partial \mathcal{L}}{\partial w_2} \quad w_0 := w_0 - \alpha \frac{\partial \mathcal{L}}{\partial w_0}$$

ML Based Classification Model

$$\text{Sigmoid} = \frac{1}{1+e^{-z}} \quad P(y = 1) = \hat{y} = \frac{1}{1+e^{-z}}$$

$$\text{Binary Cross Entropy}(\mathcal{L}) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$$



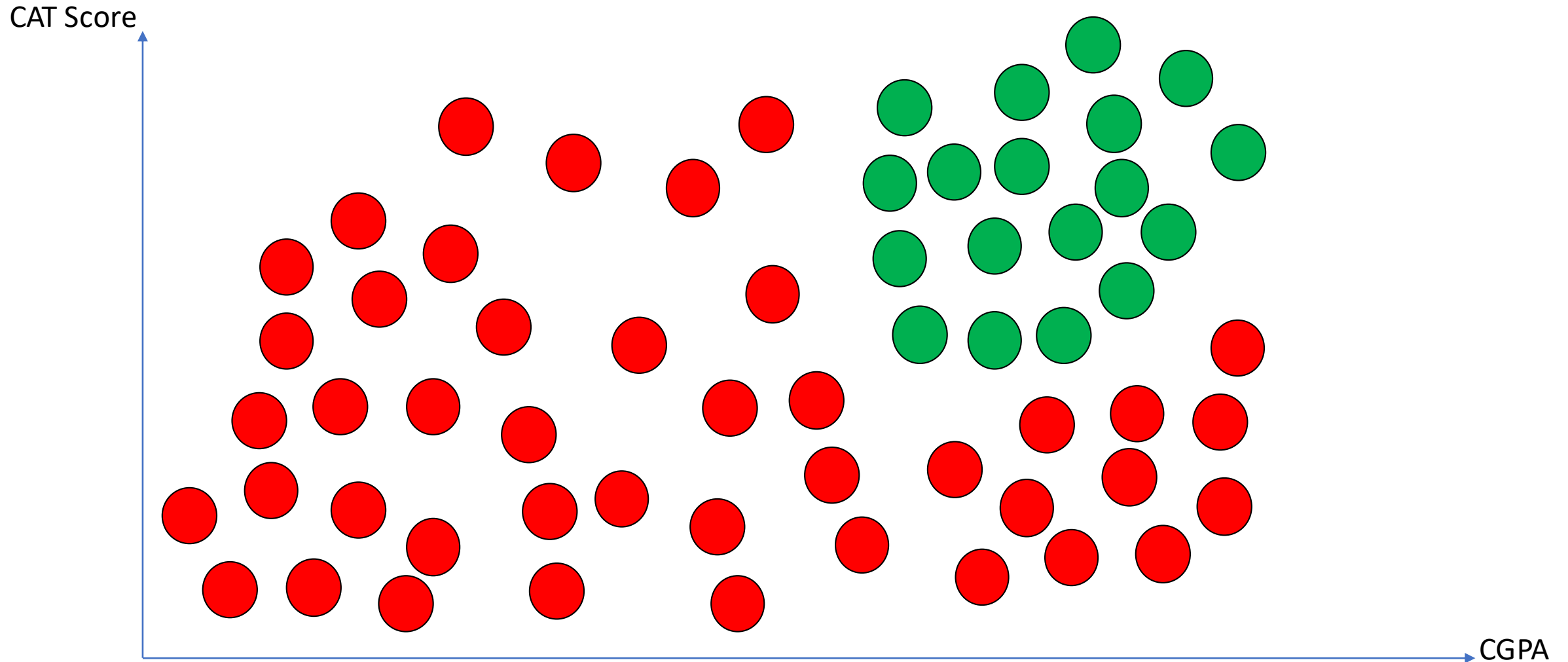
Get optimal values for W_1, W_2, W_0 by minimizing $\mathcal{L}$

Gradient Descent

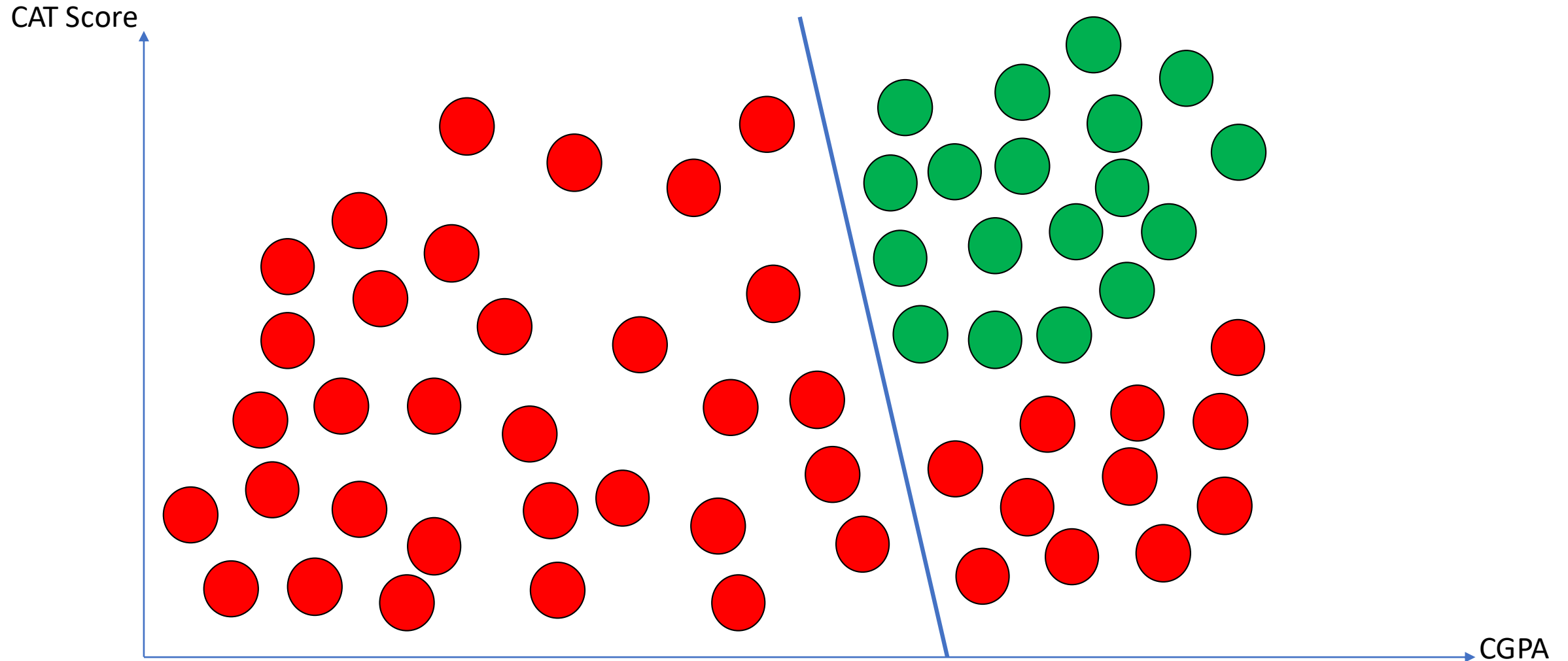
$$w_1 := w_1 - \alpha \frac{\partial \mathcal{L}}{\partial w_1} \quad w_2 := w_2 - \alpha \frac{\partial \mathcal{L}}{\partial w_2} \quad w_0 := w_0 - \alpha \frac{\partial \mathcal{L}}{\partial w_0}$$

Perceptron

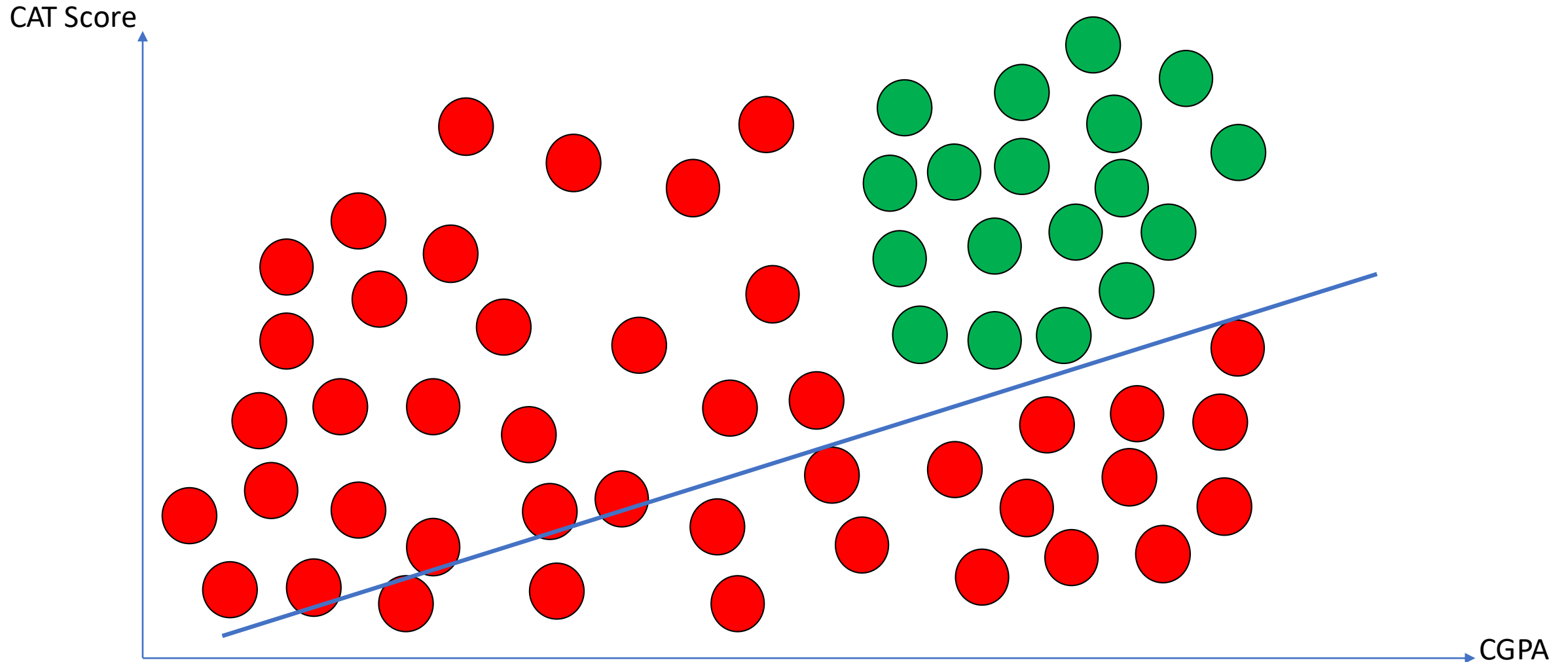
MBA admission – A complex case



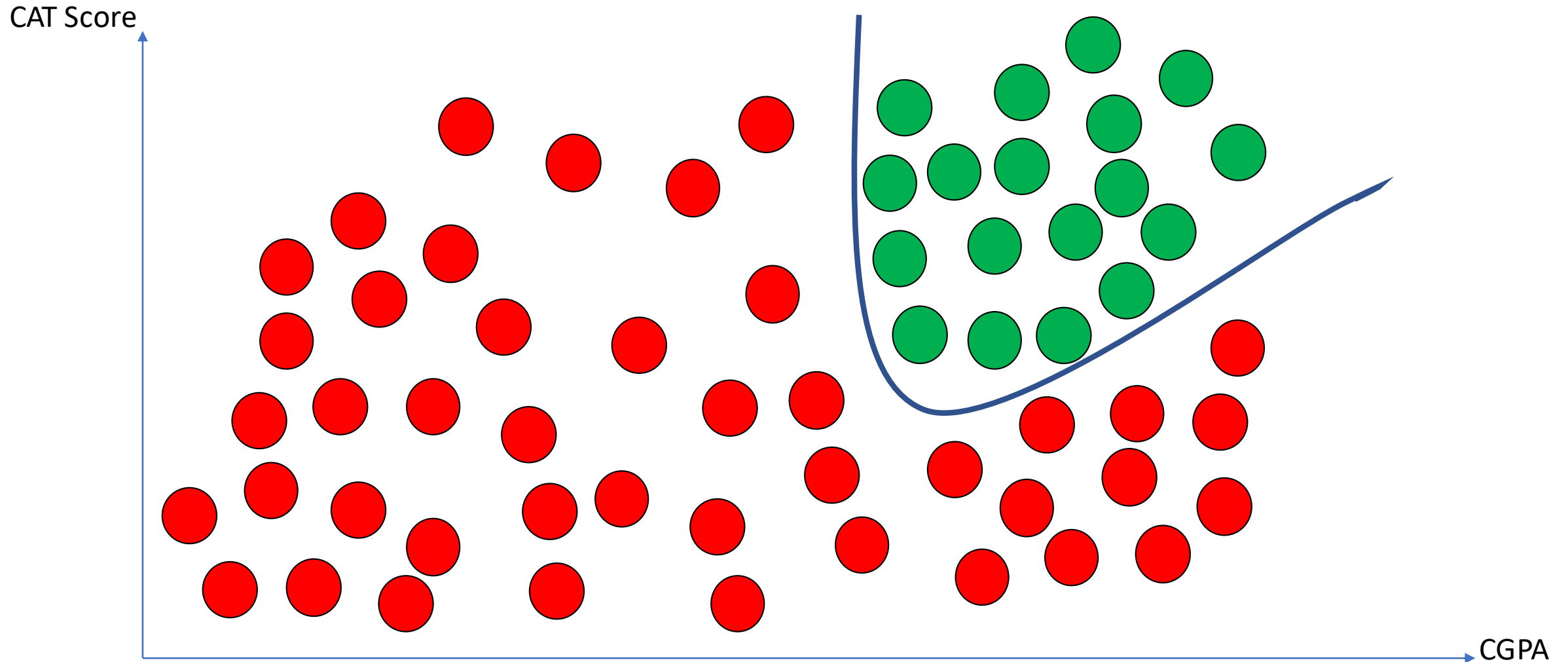
MBA admission – A complex case



MBA admission – A complex case

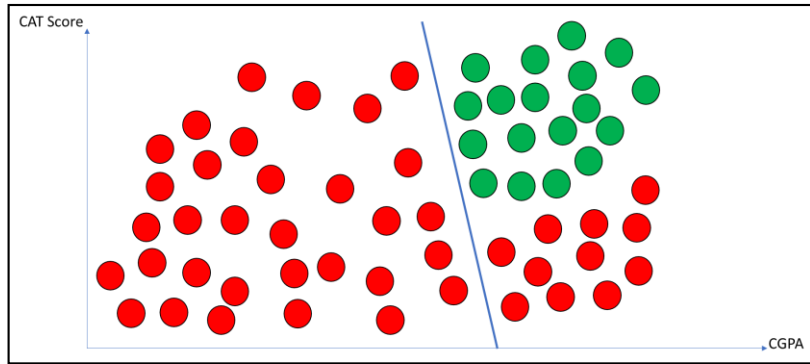


MBA admission – A complex case

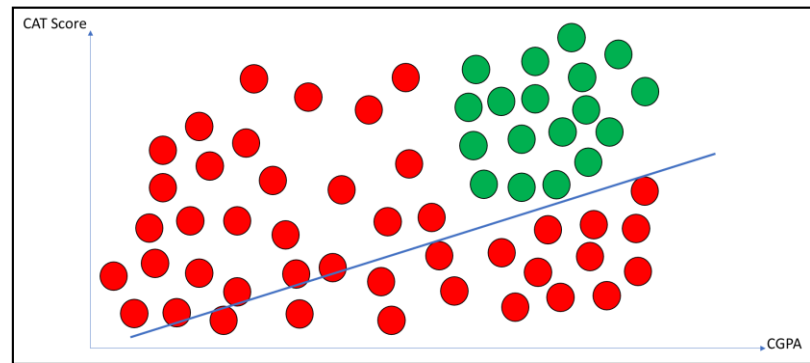
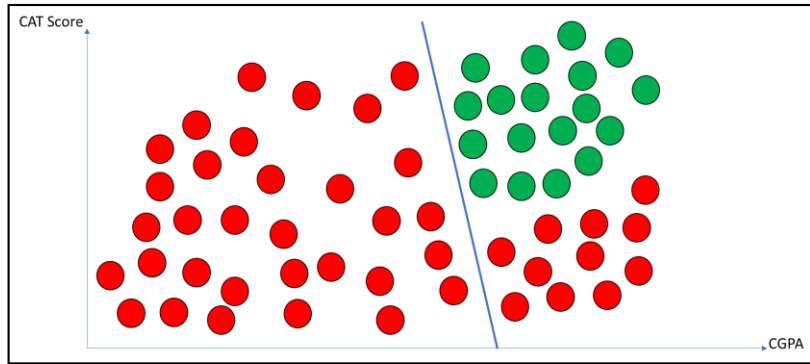


MBA admission – A complex case

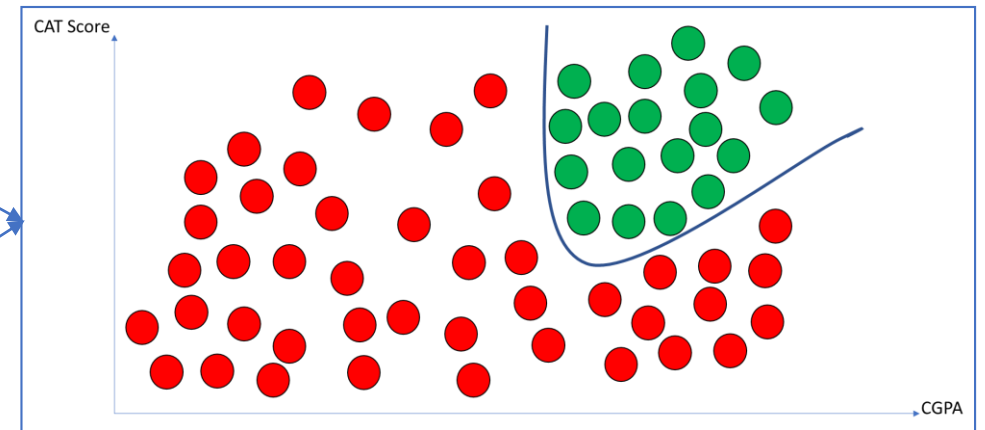
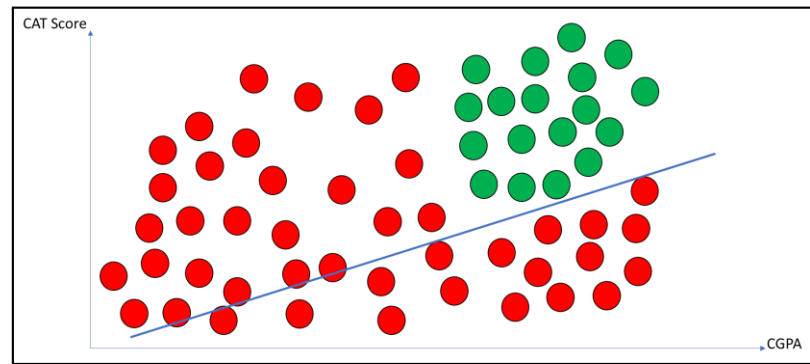
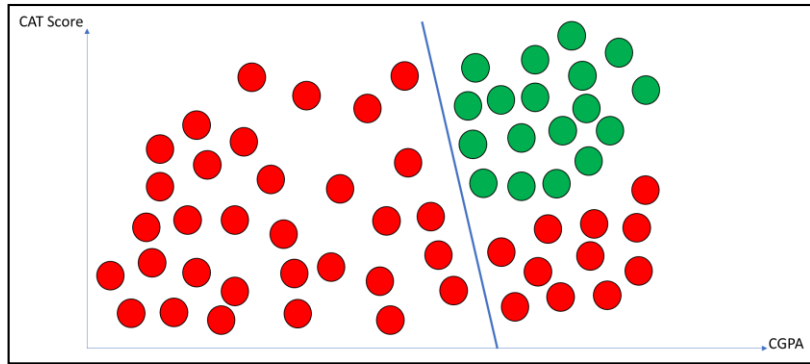
MBA admission – A complex case



MBA admission – A complex case

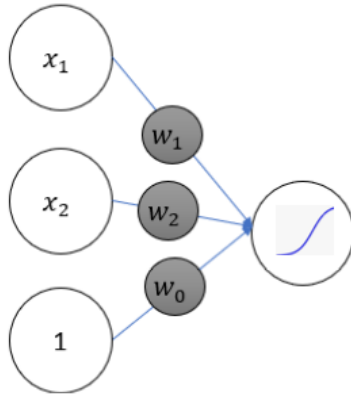


MBA admission – A complex case

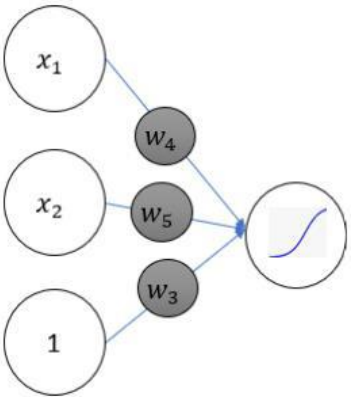
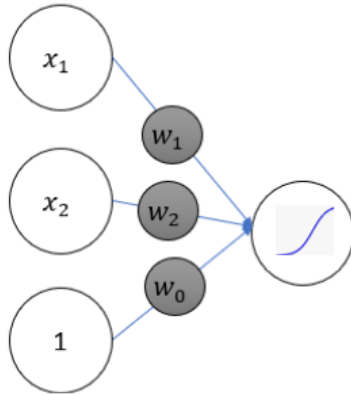


MBA admission – A complex case

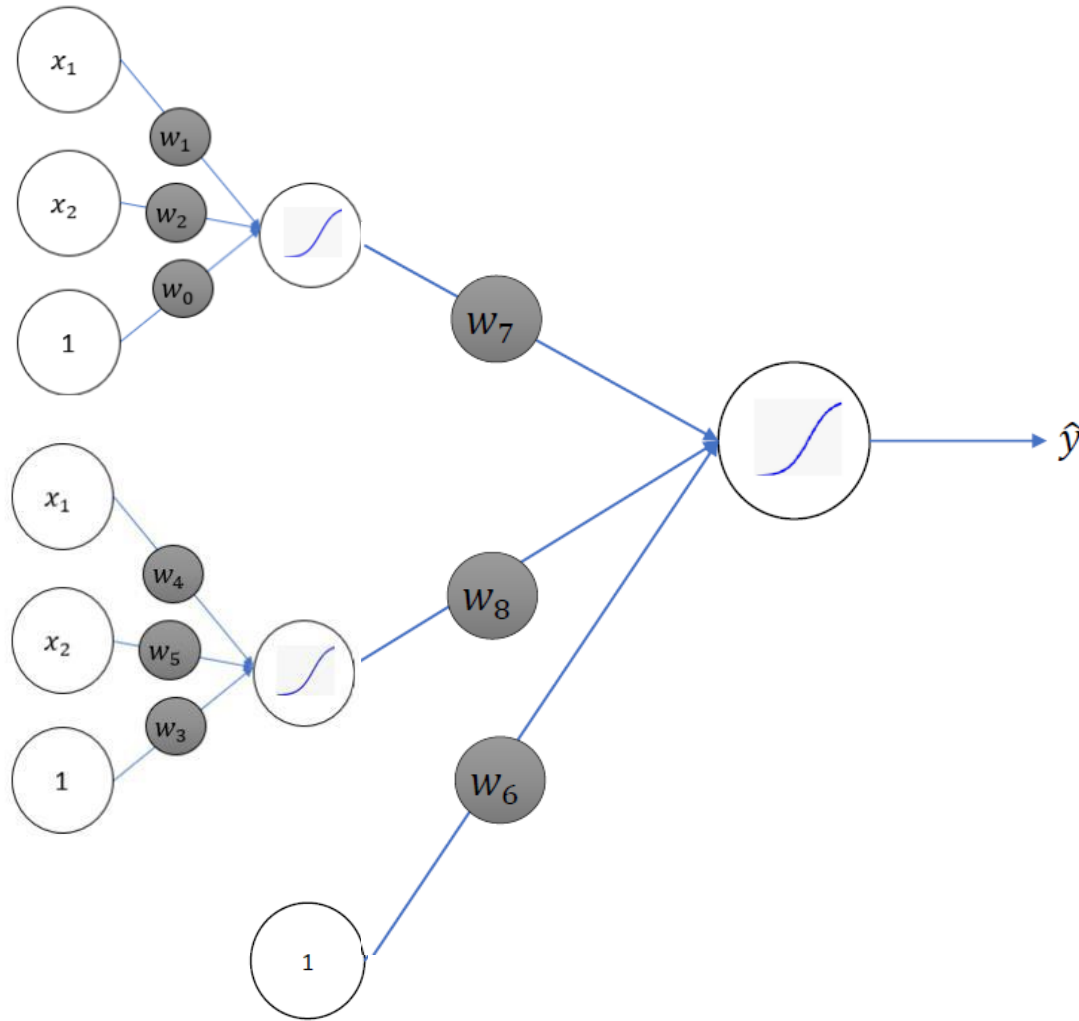
MBA admission – A complex case



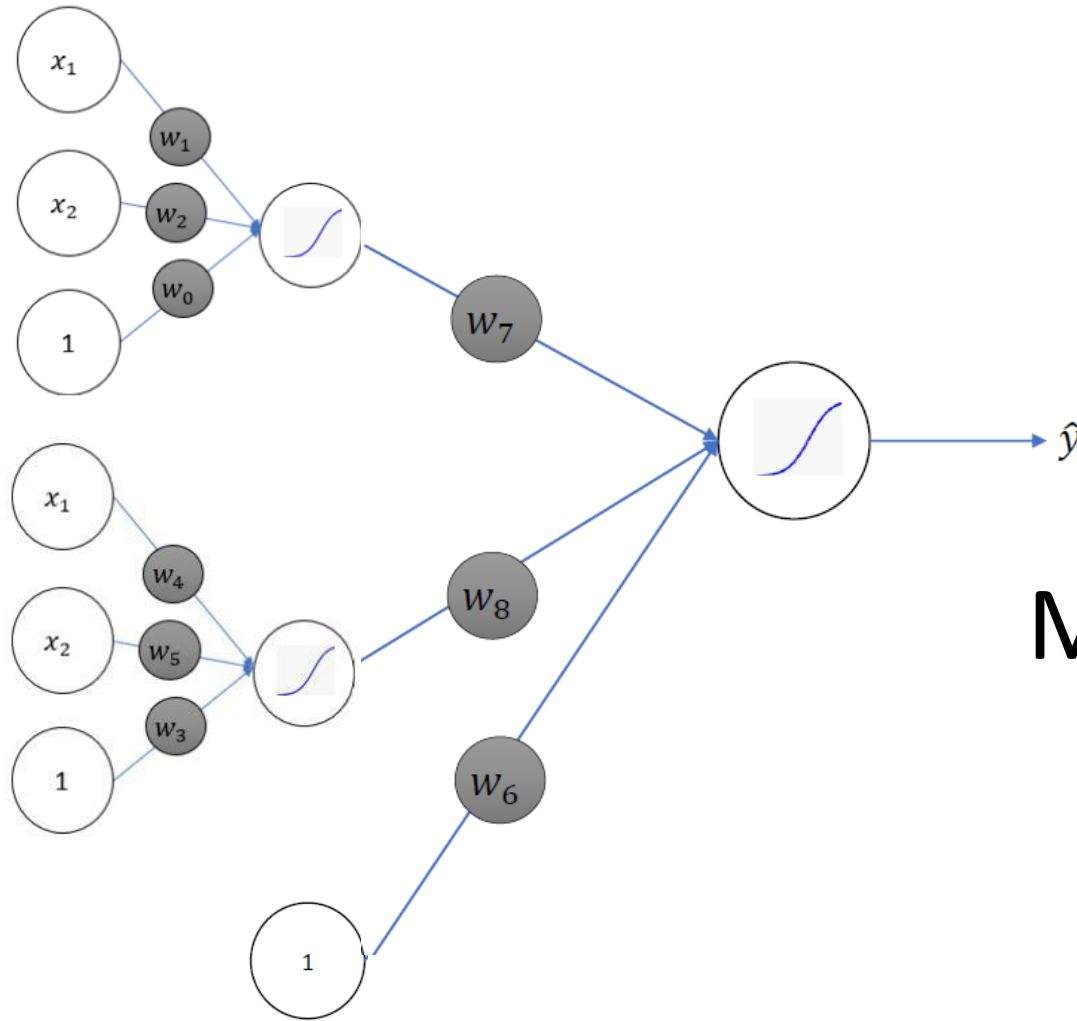
MBA admission – A complex case



MBA admission – A complex case

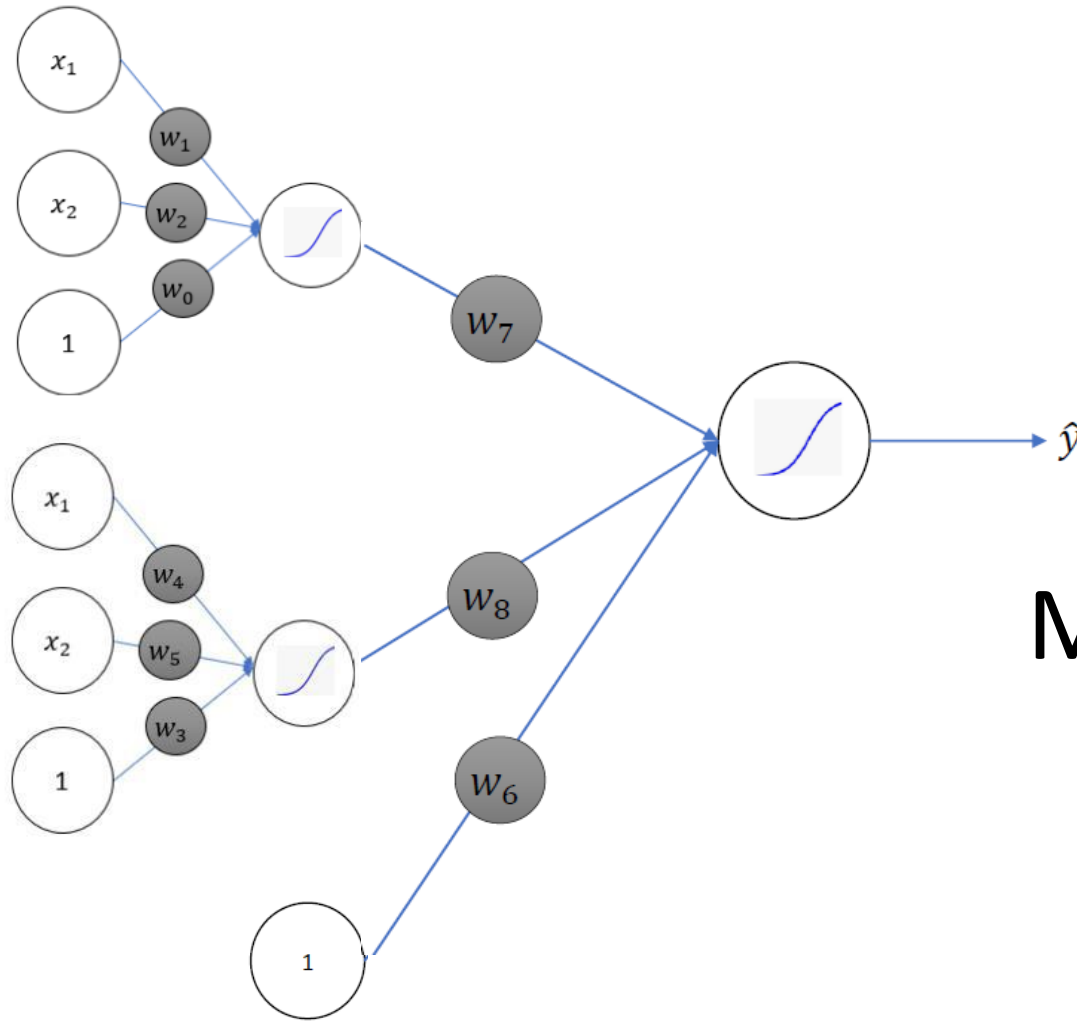


MBA admission – A complex case



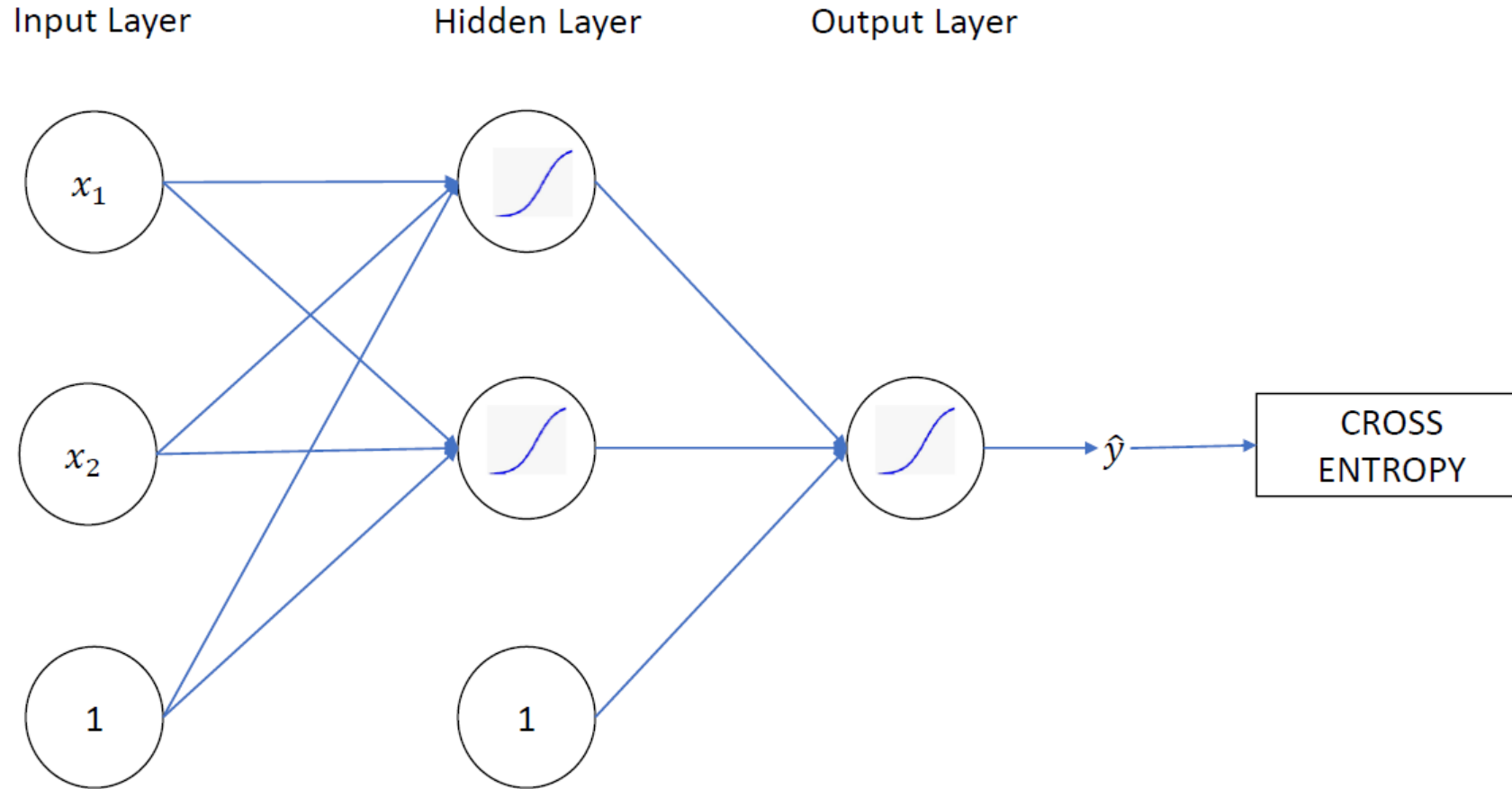
Multi Layer Perceptron

MBA admission – A complex case



Multi Layer Perceptron
Or
Neural Network

Neural Network



How do we train neural network?

How do we train neural network?

- Feed Forward

How do we train neural network?

- Feed Forward
- Back Propagation

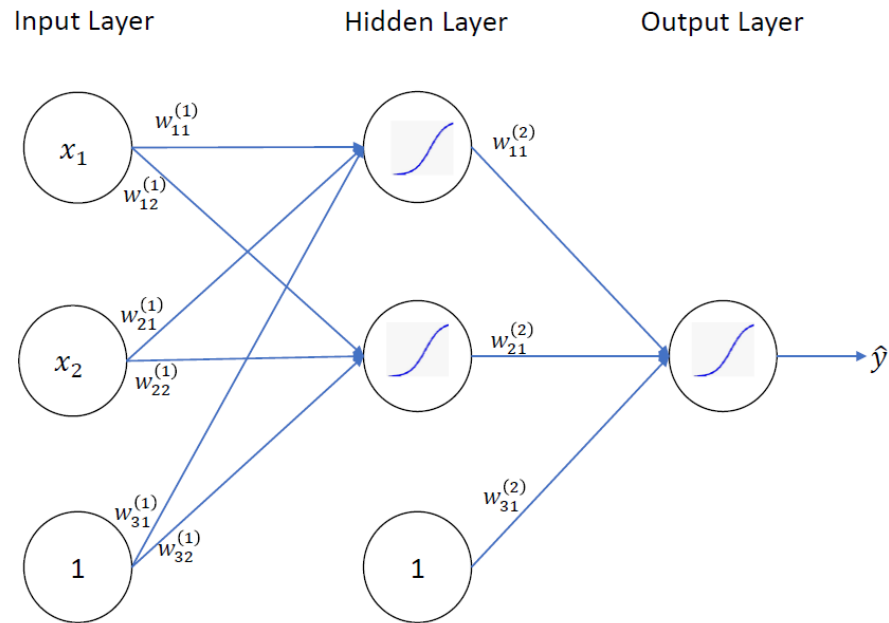
Feed Forward

Feed Forward

Turning the input into an output

Feed Forward

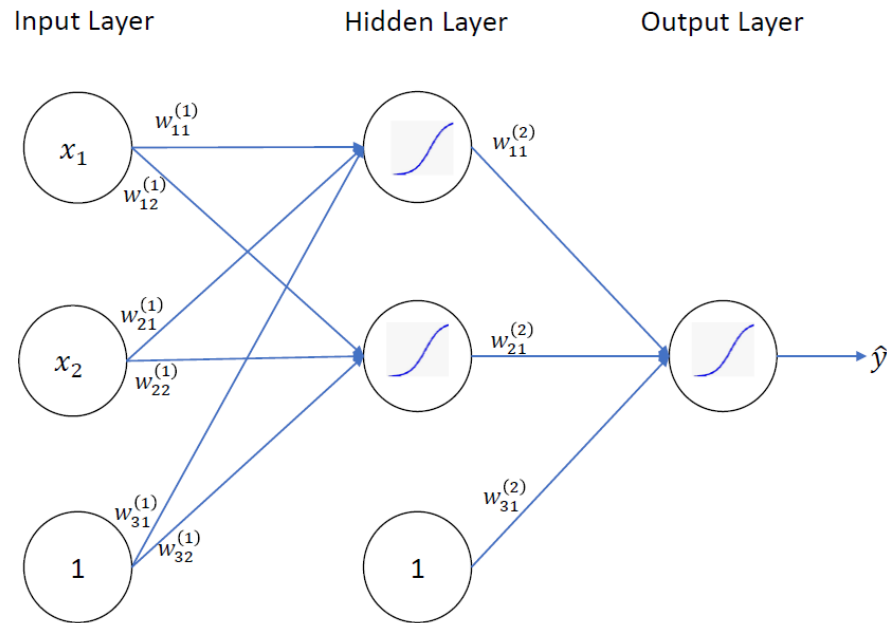
Turning the input into an output



Feed Forward

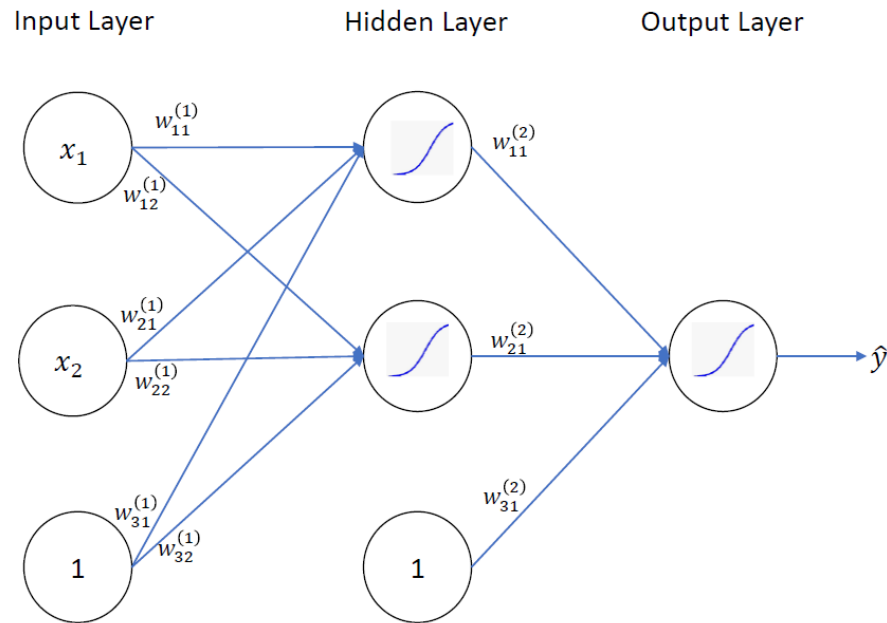
Turning the input into an output

Matrix Multiplication



Feed Forward

Turning the input into an output

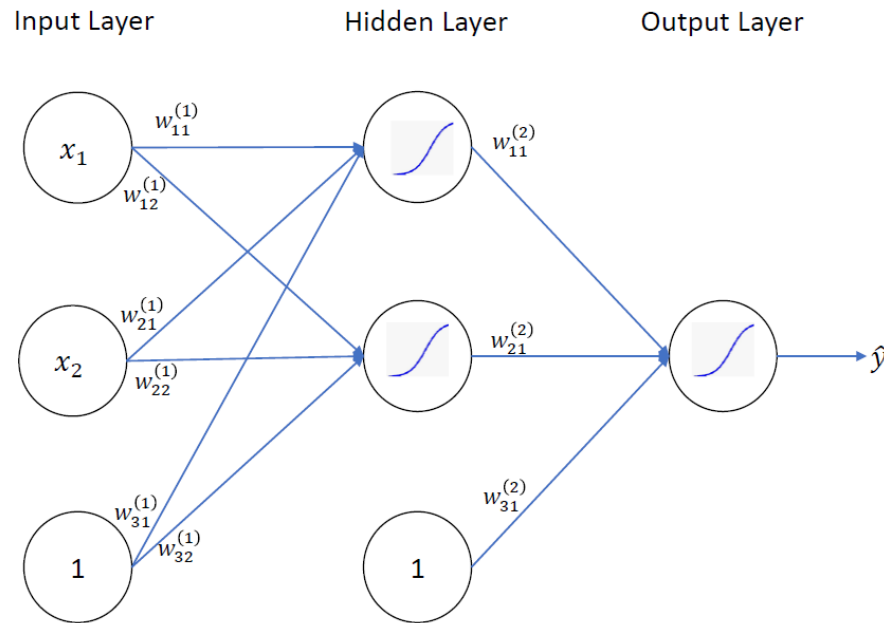


Matrix Multiplication

$$\hat{y} = \sigma \begin{pmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{pmatrix} \sigma \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix}$$

Feed Forward

Turning the input into an output



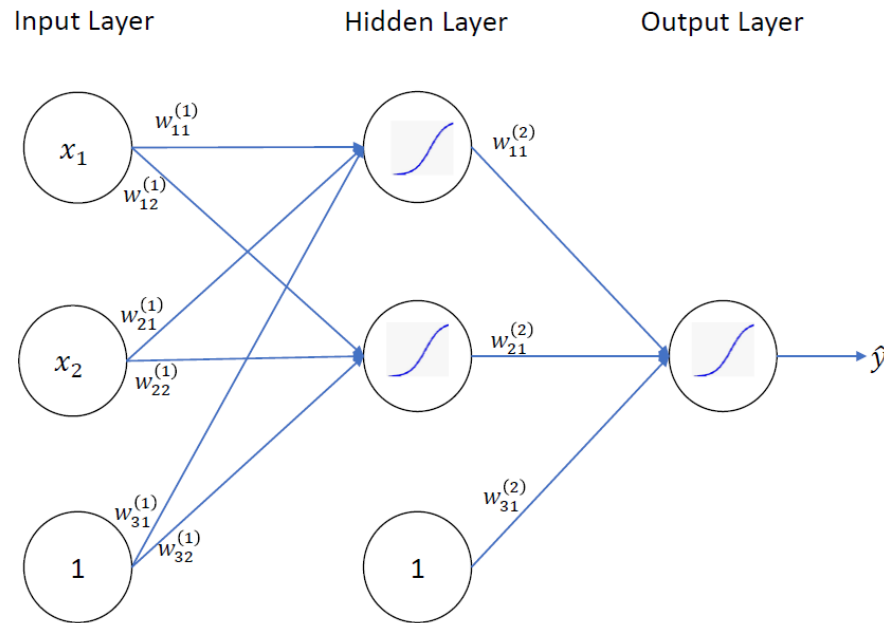
Matrix Multiplication

$$\hat{y} = \sigma \begin{pmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{pmatrix} \sigma \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix}$$

Can I use more than 2 layers?

Feed Forward

Turning the input into an output



Matrix Multiplication

$$\hat{y} = \sigma \begin{pmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{pmatrix} \sigma \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix}$$

Can I use more than 2 layers?

$$\hat{y} = \sigma \cdot w^{(3)} \cdot \sigma \cdot w^{(2)} \cdot \sigma \cdot w^{(1)} (x)$$

$$\hat{y} = \sigma \cdot w^{(n)} \cdot \dots \cdot \sigma \cdot w^{(2)} \cdot \sigma \cdot w^{(1)} (x)$$

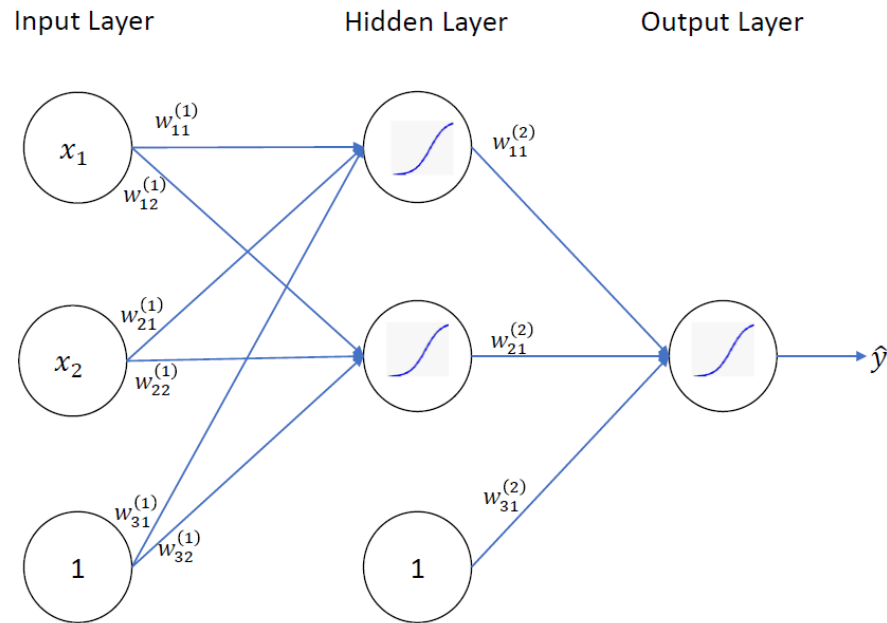
Back Propagation

Back Propagation

Distributing errors to the weights to get optimal weights

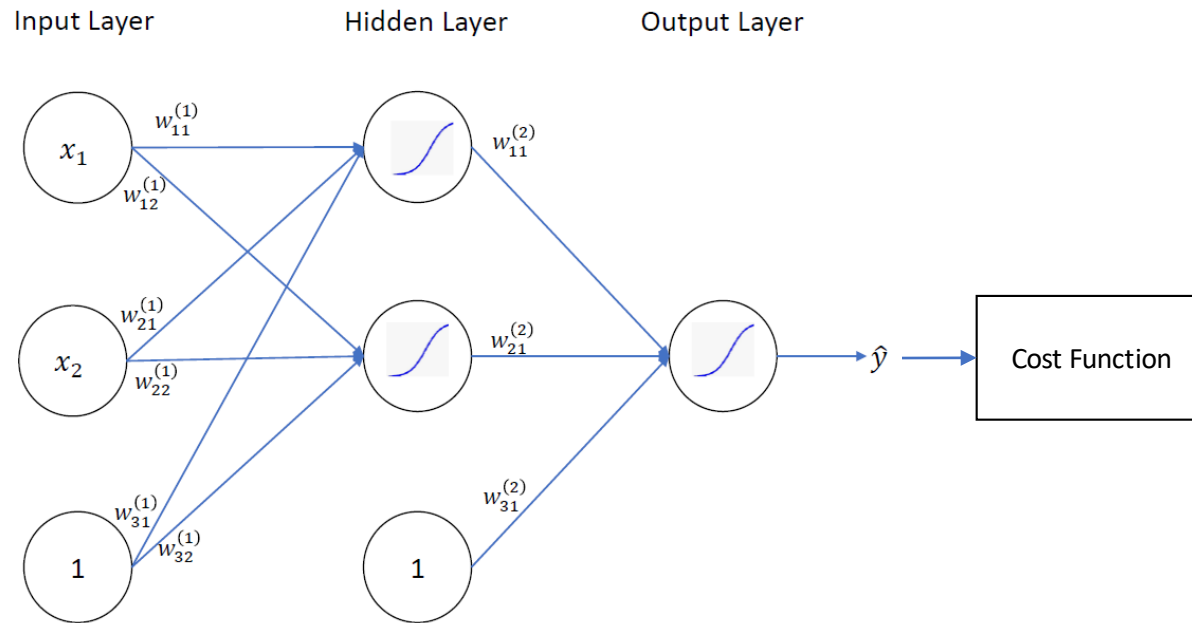
Back Propagation

Distributing errors to the weights to get optimal weights



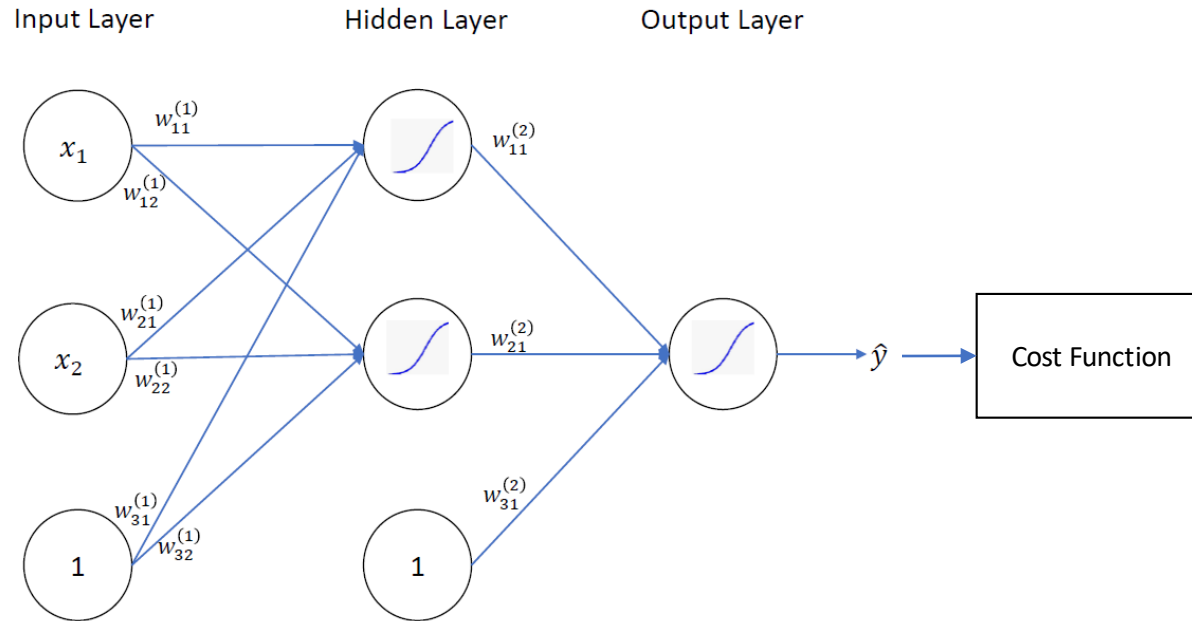
Back Propagation

Distributing errors to the weights to get optimal weights



Back Propagation

Distributing errors to the weights to get optimal weights



Prediction

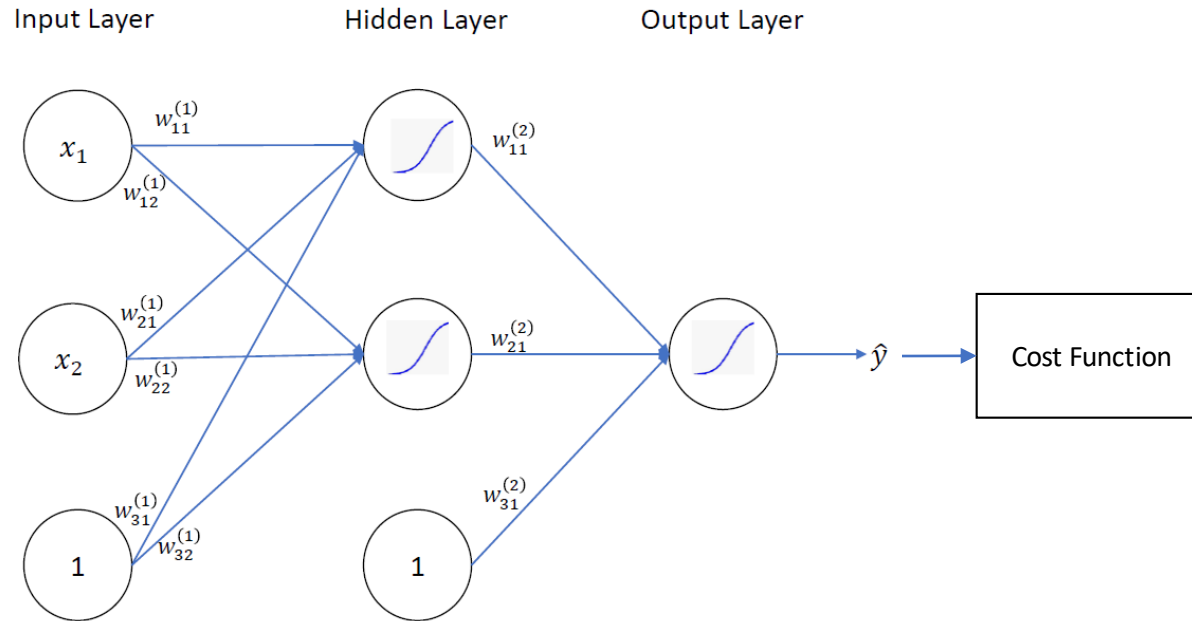
$$\hat{y} = \sigma \cdot w^{(2)} \cdot \sigma \cdot w^{(1)}(x)$$

Weights

$$w^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{pmatrix} \quad w^{(2)} = \begin{pmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{pmatrix}$$

Back Propagation

Distributing errors to the weights to get optimal weights



Prediction

$$\hat{y} = \sigma \cdot w^{(2)} \cdot \sigma \cdot w^{(1)}(x)$$

Weights

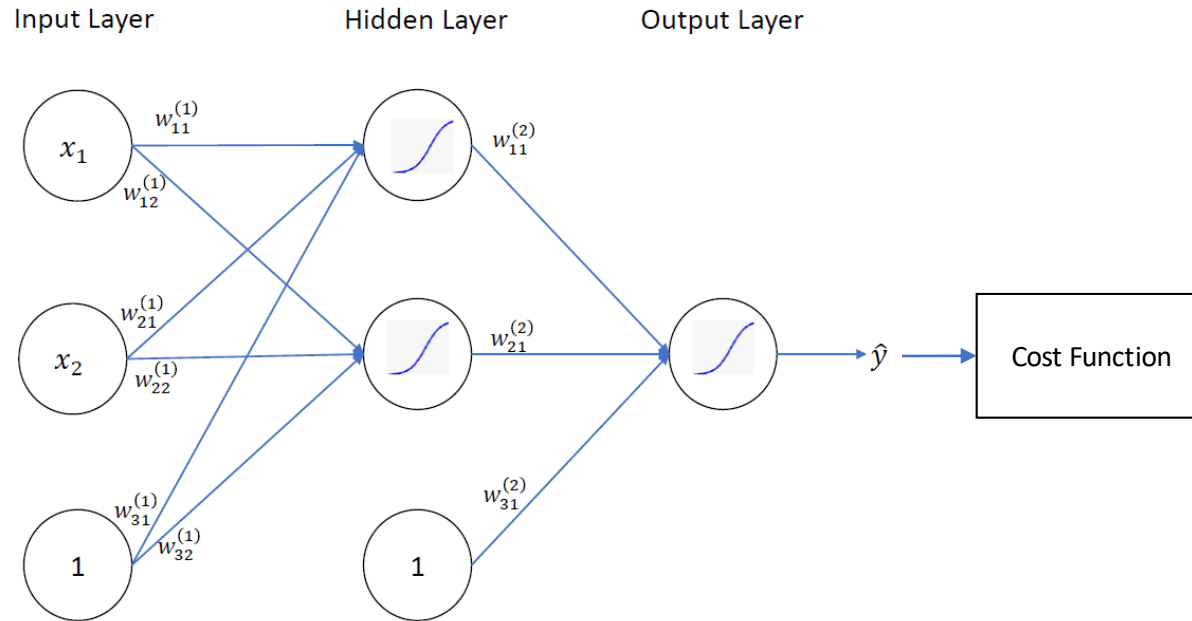
$$w^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{pmatrix} \quad w^{(2)} = \begin{pmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{pmatrix}$$

Cost Function

$$E(w) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$$

Back Propagation

Distributing errors to the weights to get optimal weights



Prediction

$$\hat{y} = \sigma \cdot w^{(2)} \cdot \sigma \cdot w^{(1)}(x)$$

Weights

$$w^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{pmatrix} \quad w^{(2)} = \begin{pmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{pmatrix}$$

Cost Function

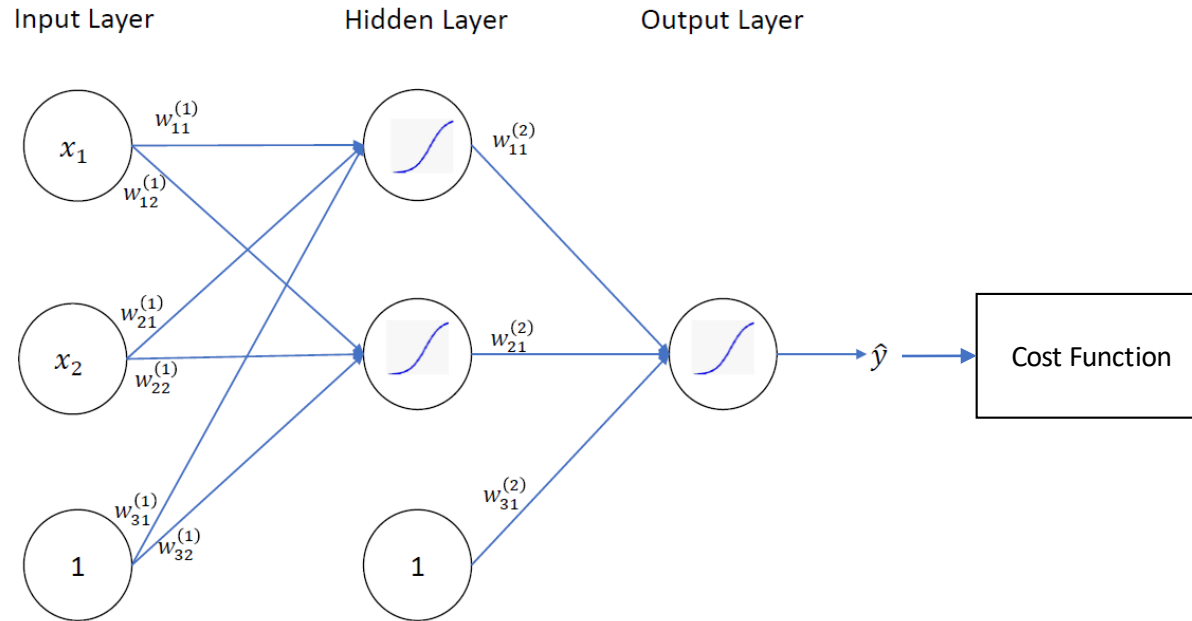
$$E(w) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$$

Gradients

$$\nabla E = \begin{pmatrix} \frac{\partial E}{\partial w_{11}^{(1)}} & \frac{\partial E}{\partial w_{12}^{(1)}} & \frac{\partial E}{\partial w_{11}^{(2)}} \\ \frac{\partial E}{\partial w_{21}^{(1)}} & \frac{\partial E}{\partial w_{22}^{(1)}} & \frac{\partial E}{\partial w_{21}^{(2)}} \\ \frac{\partial E}{\partial w_{31}^{(1)}} & \frac{\partial E}{\partial w_{32}^{(1)}} & \frac{\partial E}{\partial w_{31}^{(2)}} \end{pmatrix}$$

Back Propagation

Distributing errors to the weights to get optimal weights



Prediction

$$\hat{y} = \sigma \cdot w^{(2)} \cdot \sigma \cdot w^{(1)}(x)$$

Weights

$$w^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{pmatrix} \quad w^{(2)} = \begin{pmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{pmatrix}$$

Cost Function

$$E(w) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$$

Gradients

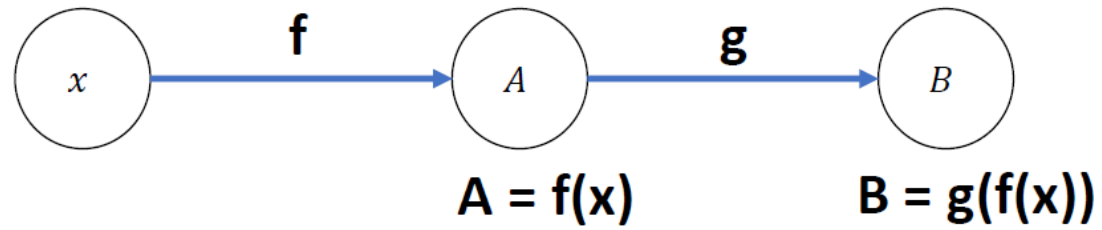
$$\nabla E = \begin{pmatrix} \frac{\partial E}{\partial w_{11}^{(1)}} & \frac{\partial E}{\partial w_{12}^{(1)}} & \frac{\partial E}{\partial w_{11}^{(2)}} \\ \frac{\partial E}{\partial w_{21}^{(1)}} & \frac{\partial E}{\partial w_{22}^{(1)}} & \frac{\partial E}{\partial w_{21}^{(2)}} \\ \frac{\partial E}{\partial w_{31}^{(1)}} & \frac{\partial E}{\partial w_{32}^{(1)}} & \frac{\partial E}{\partial w_{31}^{(2)}} \end{pmatrix}$$

Weight updates

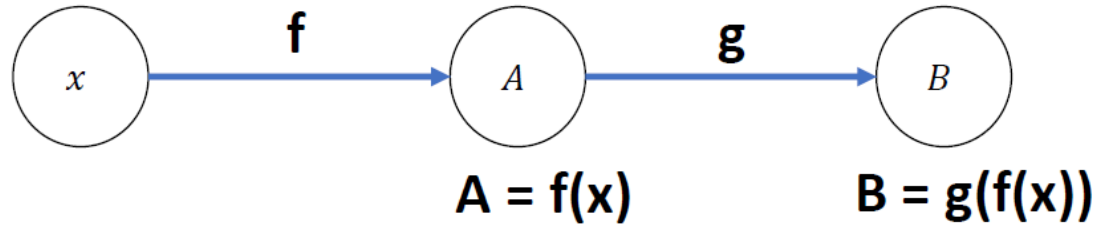
$$w_{ij}^{(k)} := w_{ij}^{(k)} - \alpha \frac{\partial E}{\partial w_{ij}^{(k)}}$$

Vanishing Gradients

Vanishing Gradients

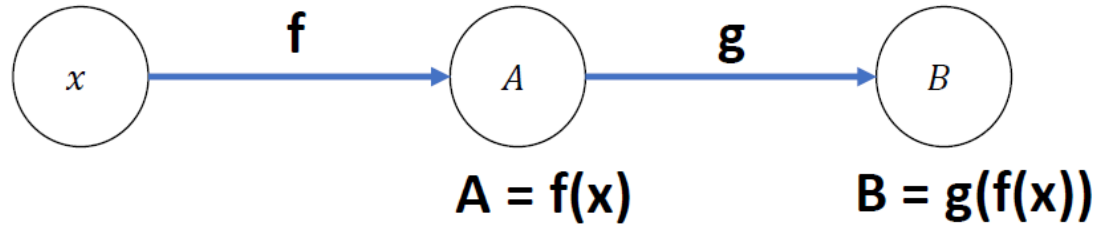


Vanishing Gradients

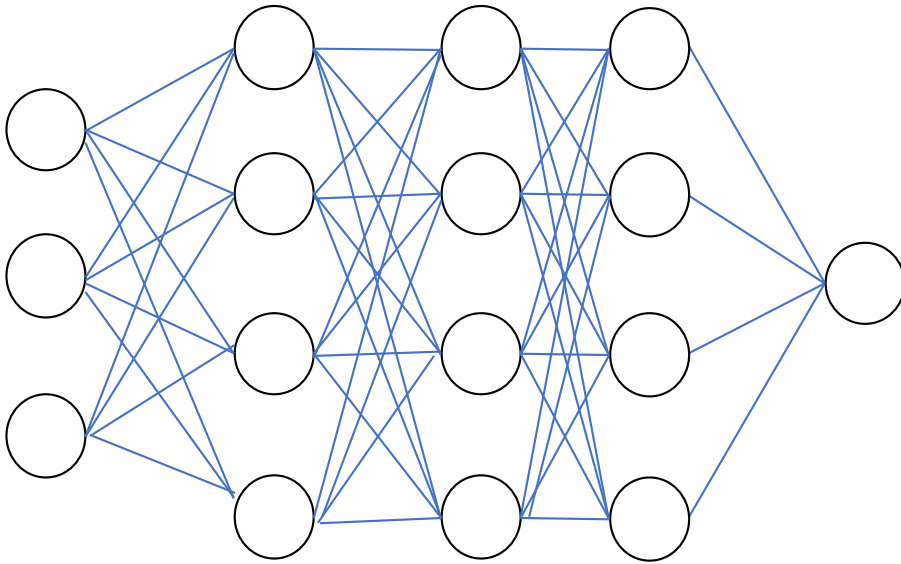


$$\frac{\partial B}{\partial x} = \frac{\partial A}{\partial x} \frac{\partial B}{\partial A}$$

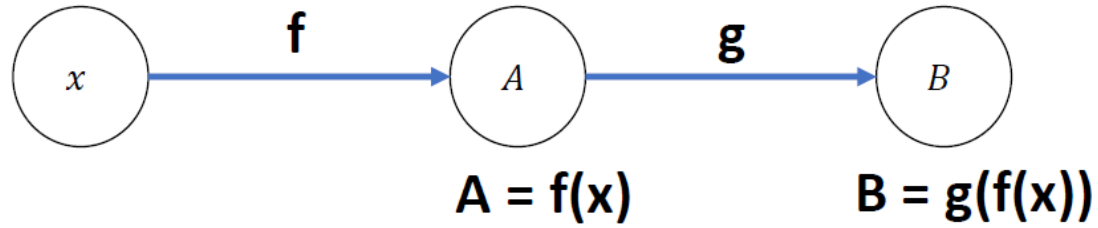
Vanishing Gradients



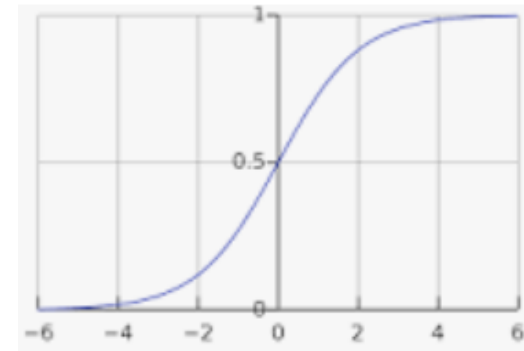
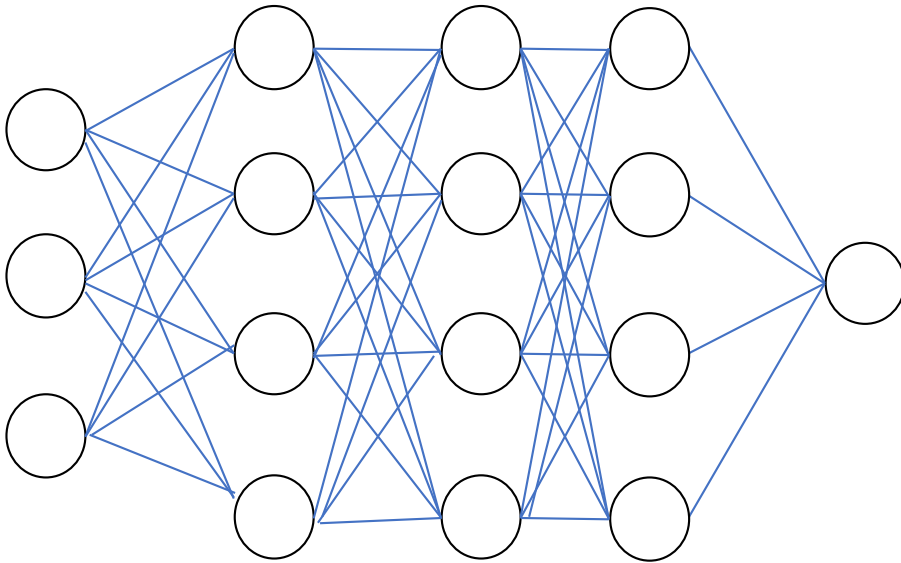
$$\frac{\partial B}{\partial x} = \frac{\partial A}{\partial x} \frac{\partial B}{\partial A}$$



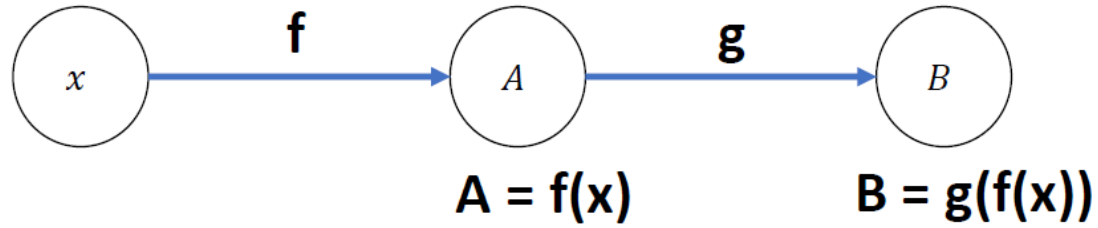
Vanishing Gradients



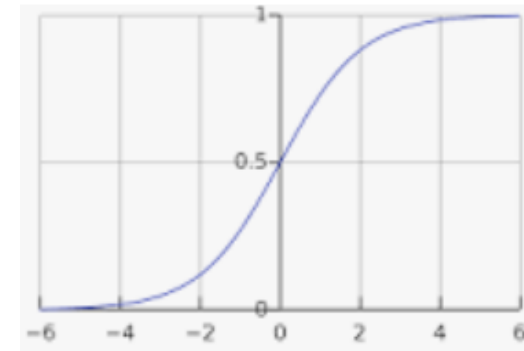
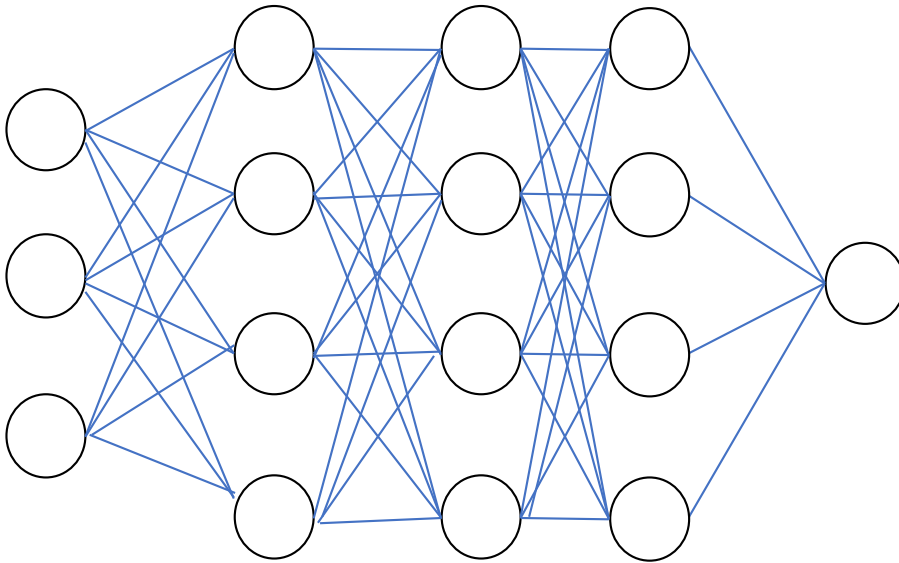
$$\frac{\partial B}{\partial x} = \frac{\partial A}{\partial x} \frac{\partial B}{\partial A}$$



Vanishing Gradients

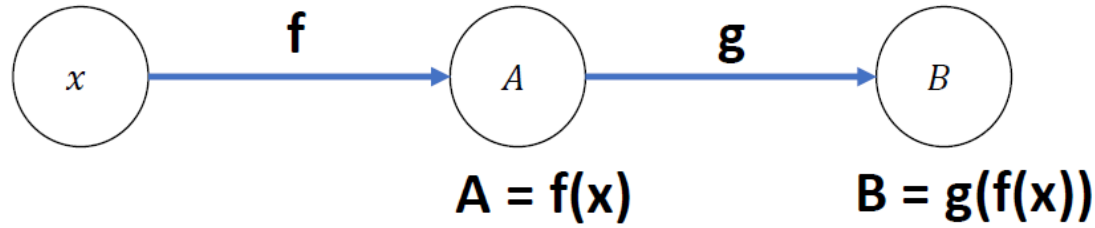


$$\frac{\partial B}{\partial x} = \frac{\partial A}{\partial x} \frac{\partial B}{\partial A}$$

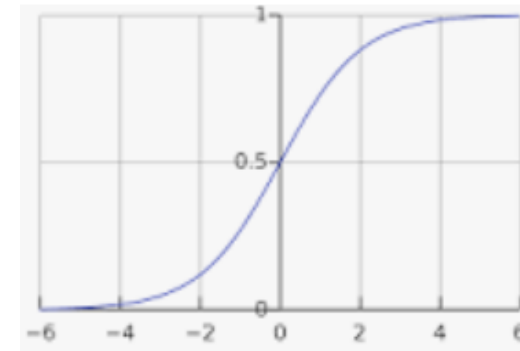
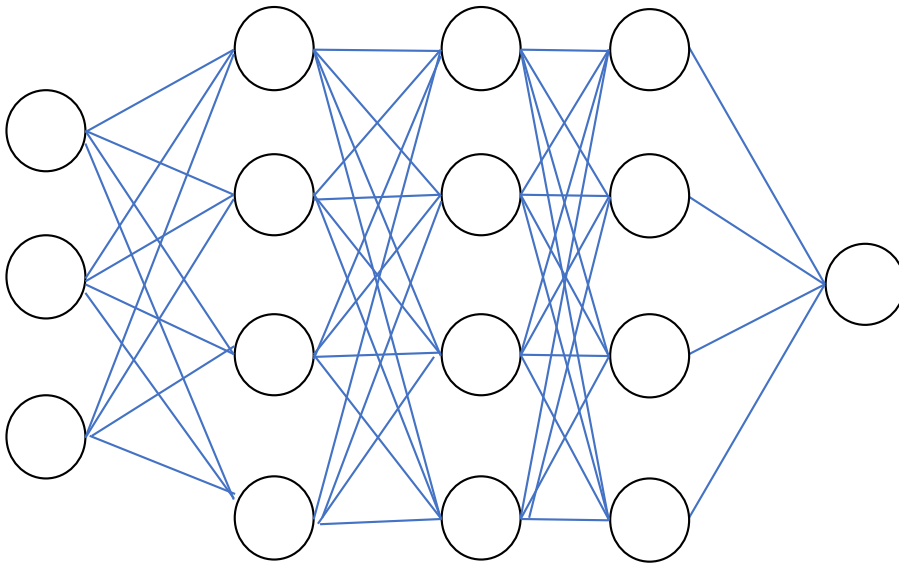


Gradient values are
between 0 to 0.25

Vanishing Gradients



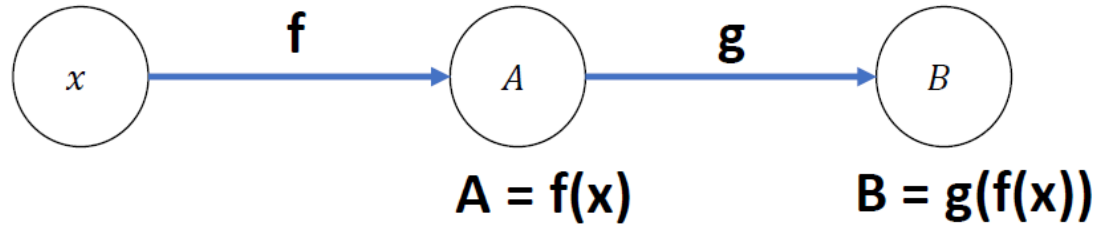
$$\frac{\partial B}{\partial x} = \frac{\partial A}{\partial x} \frac{\partial B}{\partial A}$$



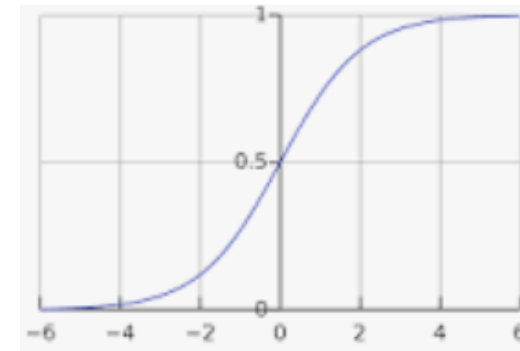
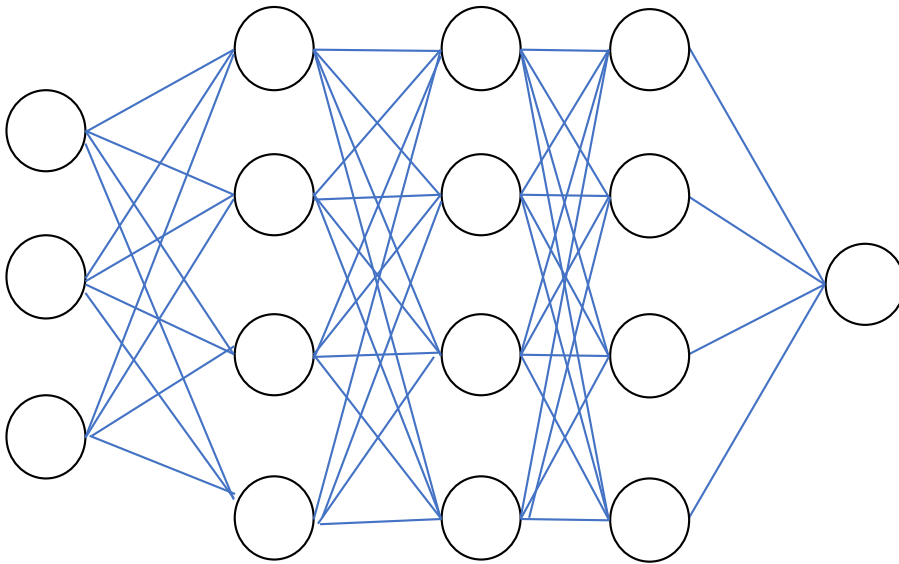
Gradient values are between 0 to 0.25

$$\frac{\partial E}{\partial w_{11}} = \text{Gradient1} \times \text{Gradient2} \times \text{Gradient3} \dots \dots \text{GradientN}$$

Vanishing Gradients



$$\frac{\partial B}{\partial x} = \frac{\partial A}{\partial x} \frac{\partial B}{\partial A}$$



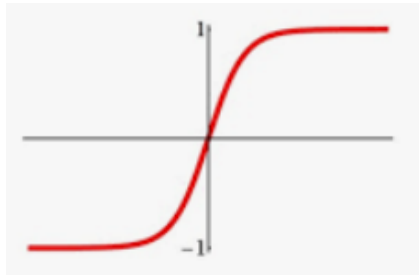
Gradient values are between 0 to 0.25

$$\frac{\partial E}{\partial w_{11}} = \text{Gradient1} \times \text{Gradient2} \times \text{Gradient3} \dots \text{GradientN} = \text{A value which is close to zero}$$

Vanishing Gradients – What do we do?

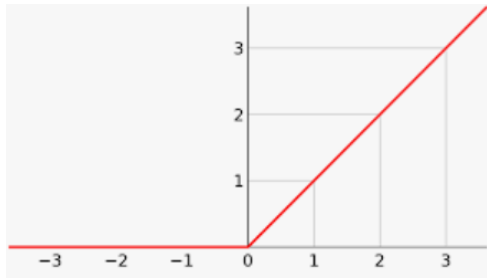
Vanishing Gradients – What do we do?

Other activation functions



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

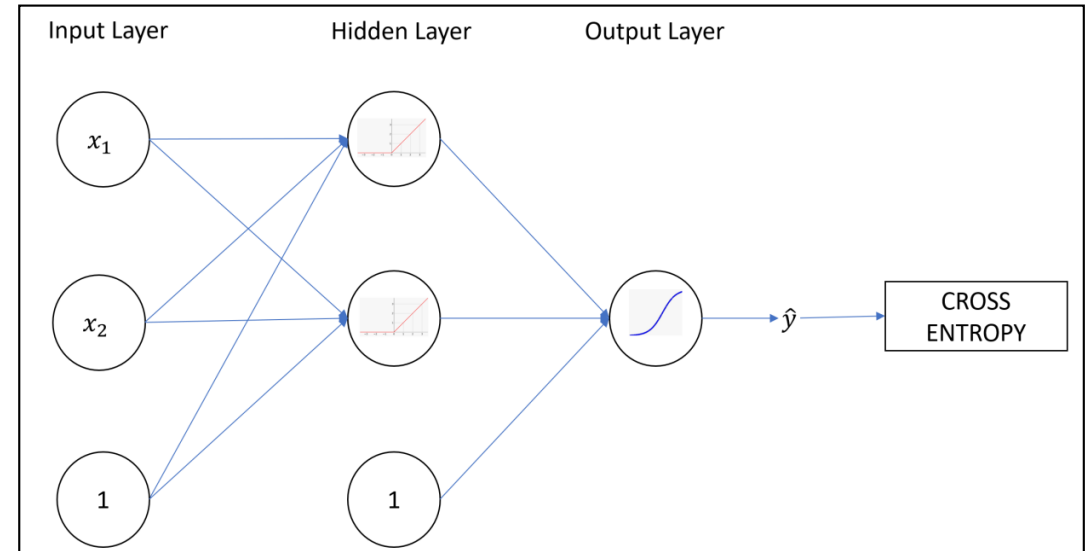
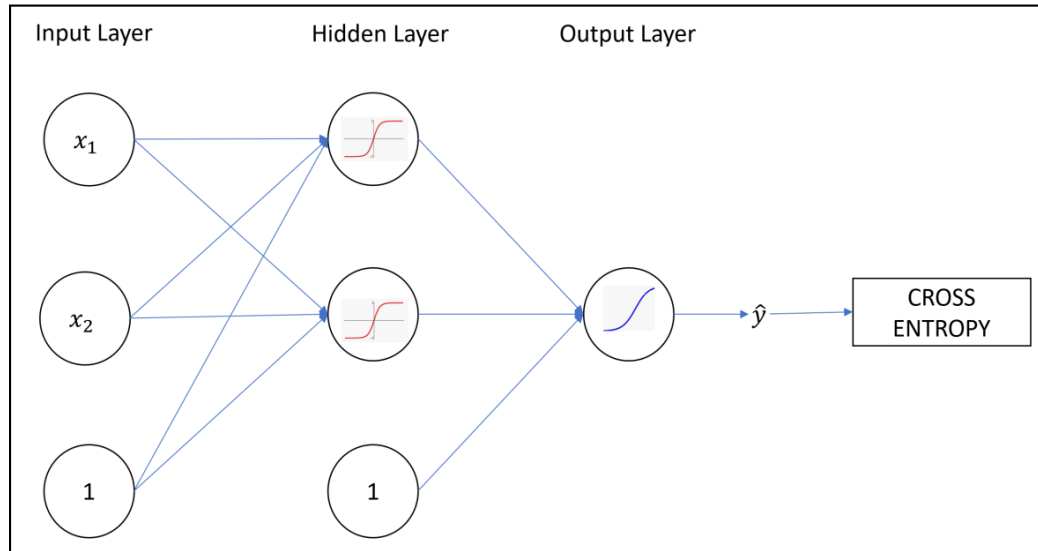
Similar to sigmoid. But, since the range is between -1 to +1 the derivatives are larger



$$R(x) = \max(0, x)$$

Derivative is 1 if the number is positive

Vanishing Gradients – What do we do?

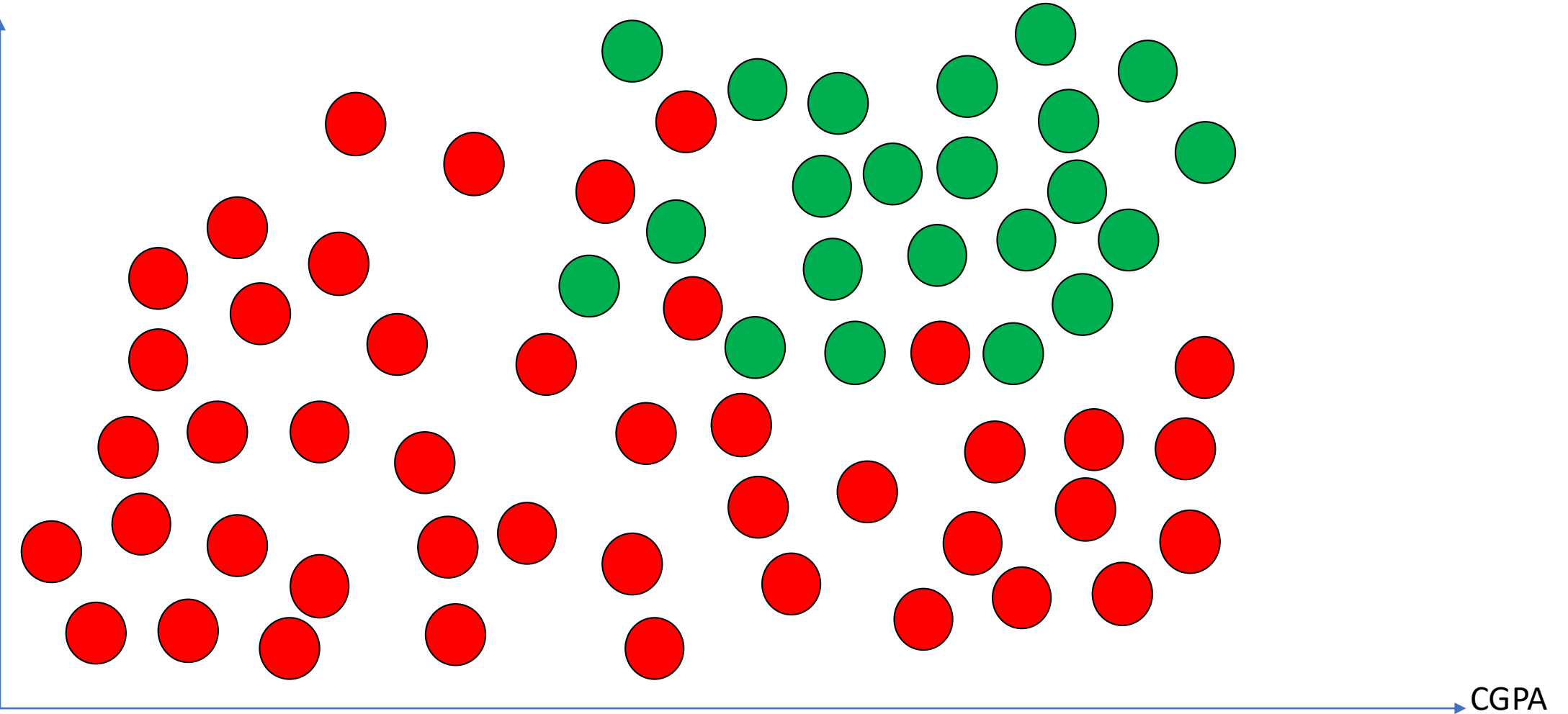


- Only the hidden layer neurons have been changed
- The output layer neuron will be determined based on the variable and number of classes

Overfitting

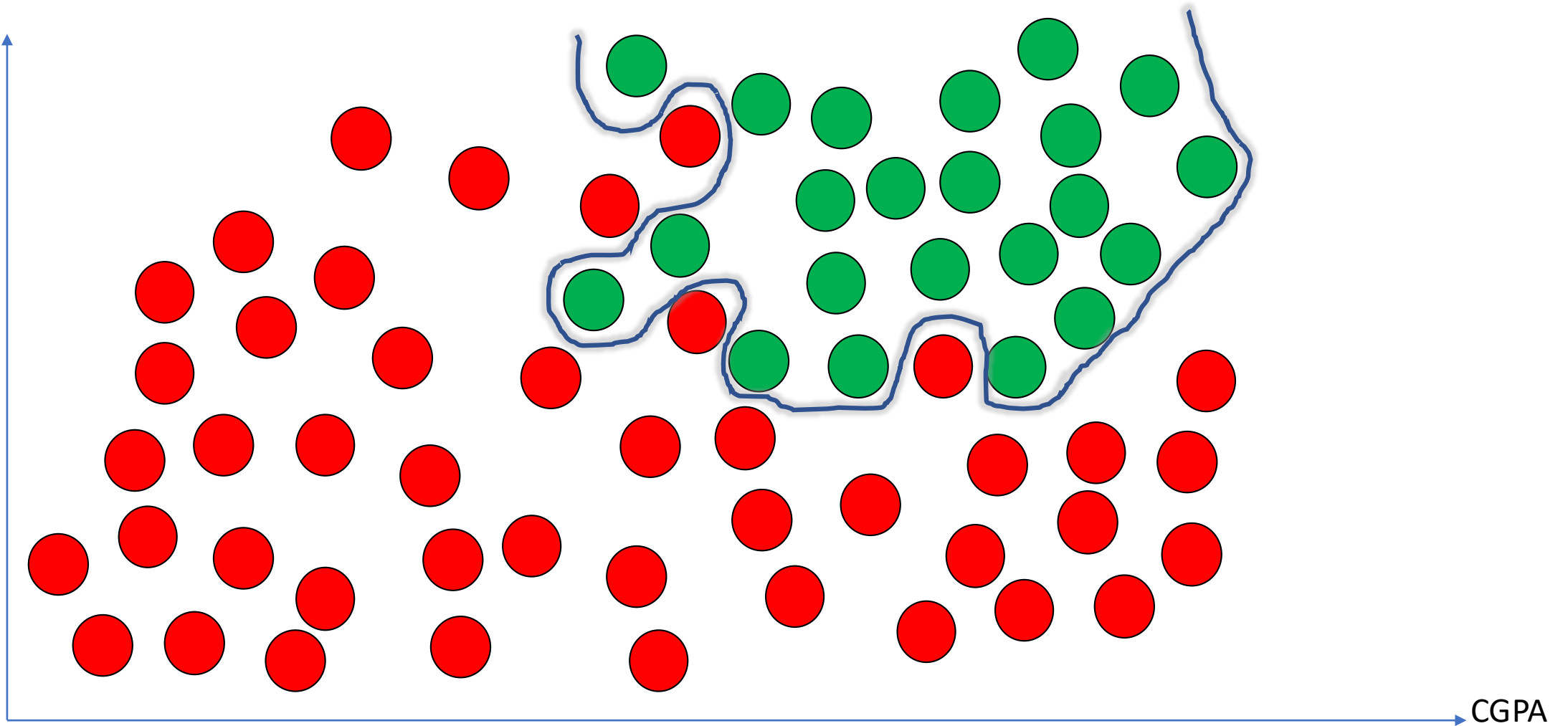
Overfitting

CAT Score



Overfitting

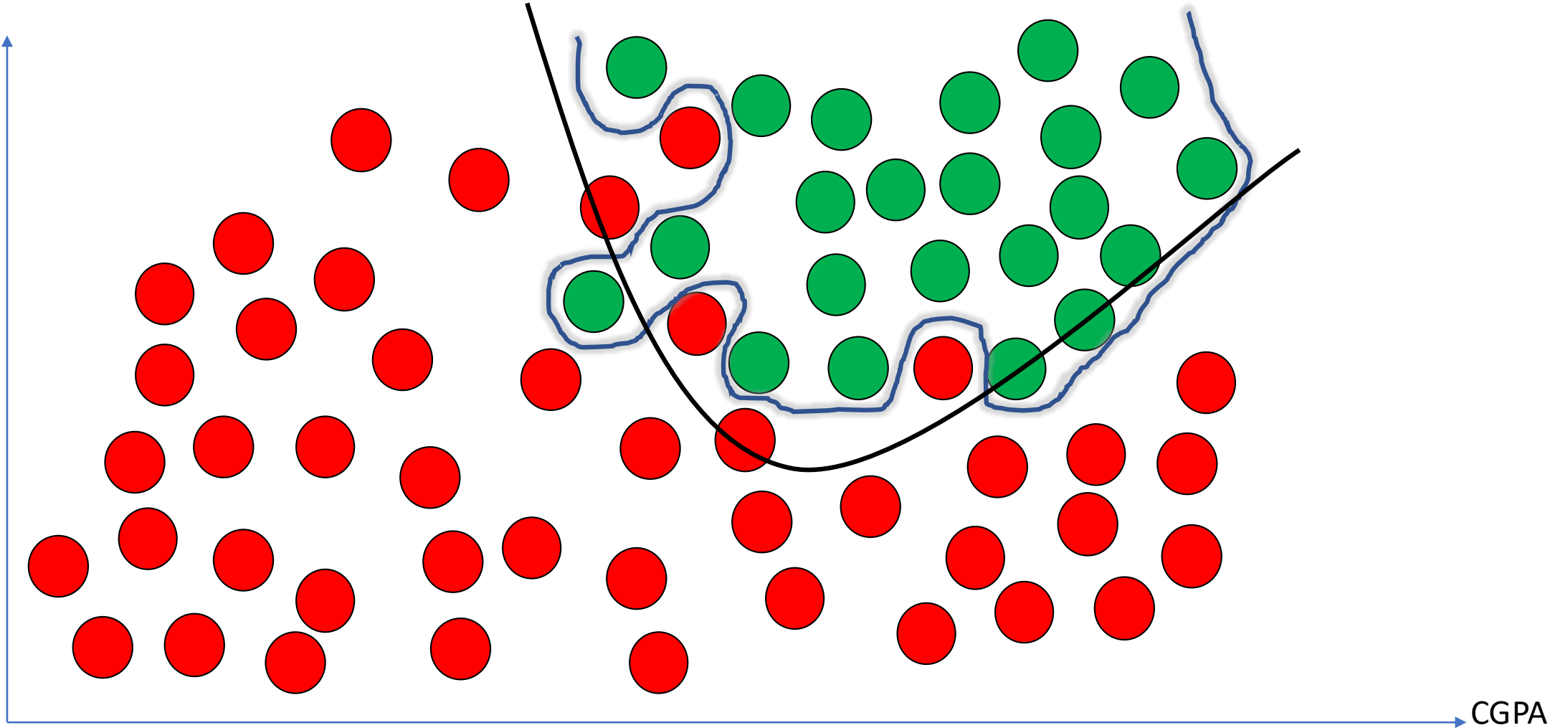
CAT Score



CGPA

Overfitting

CAT Score



CGPA

Overfitting – What do we do?

Overfitting – What do we do?

Regularization

Overfitting – What do we do?

Regularization

1. Create complex model which will overfit
2. Attach a regularize component to cost function to get a robust model

Overfitting – What do we do?

Regularization

1. Create complex model which will overfit
2. Attach a regularize component to cost function to get a robust model

L1 Regularization

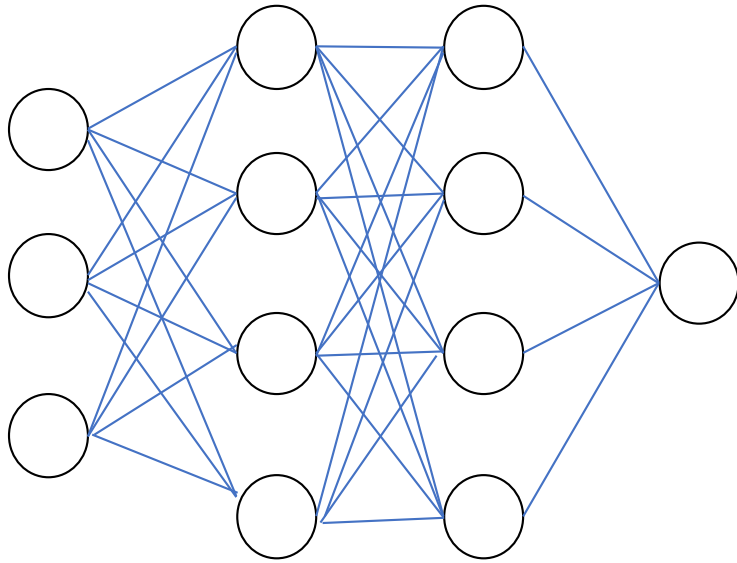
$$E(w) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) + \lambda \sum_{i=1}^n |w_i|$$

L2 Regularization

$$E(w) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) + \lambda \sum_{i=1}^n w_i^2$$

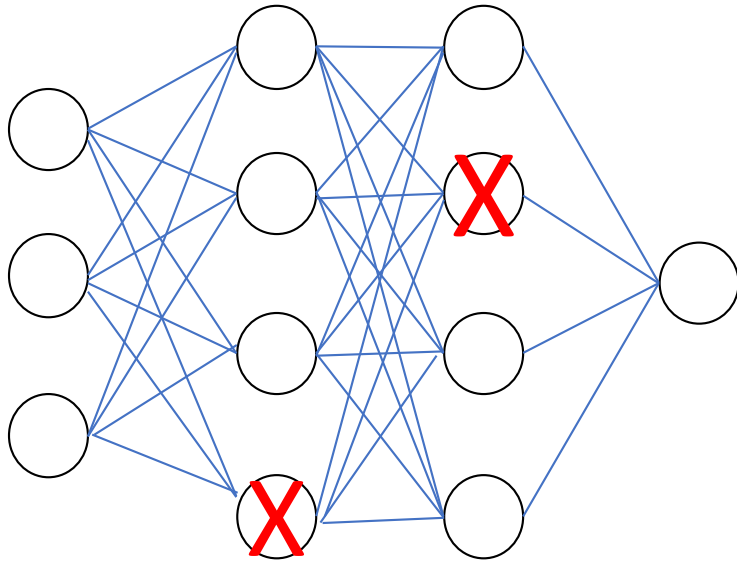
Overfitting – What do we do?

DROP OUT



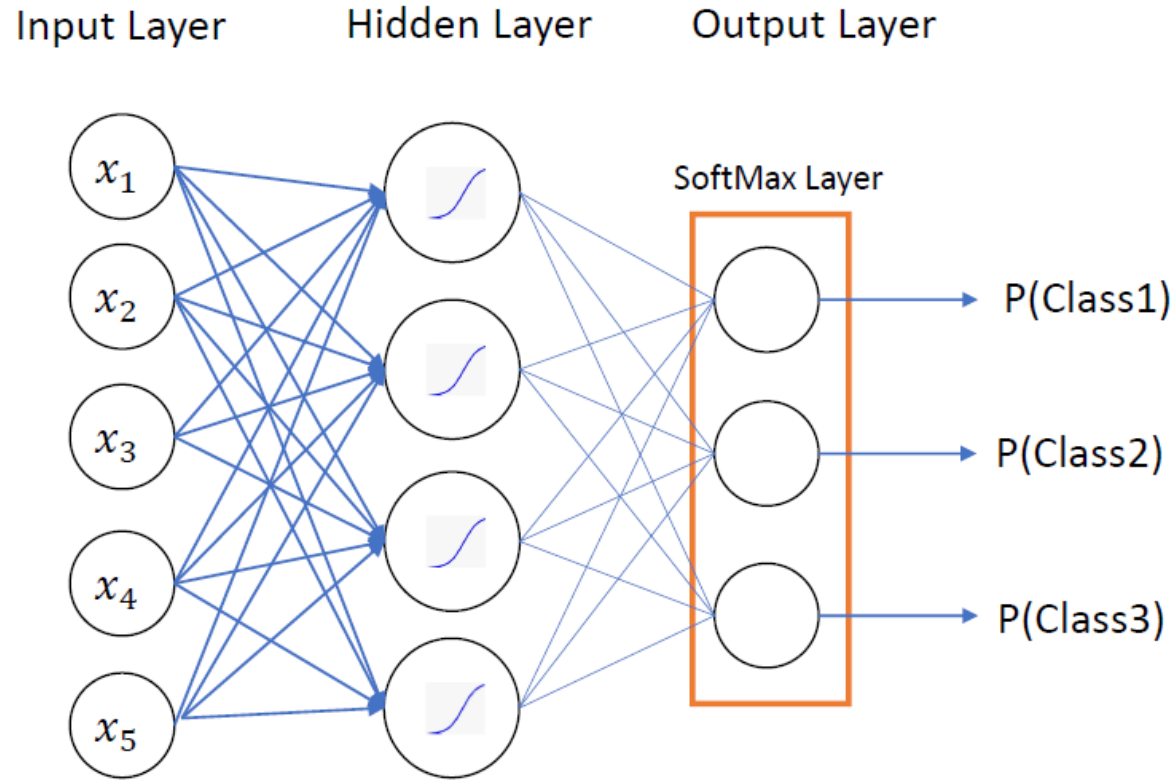
Overfitting – What do we do?

DROP OUT



- Drop out probability = p
- A portion of the neurons are deactivated.
- Dropout prevents neurons from **co-adapting** too much
- This happens only during training.

Neural Network for Multi Class Problem

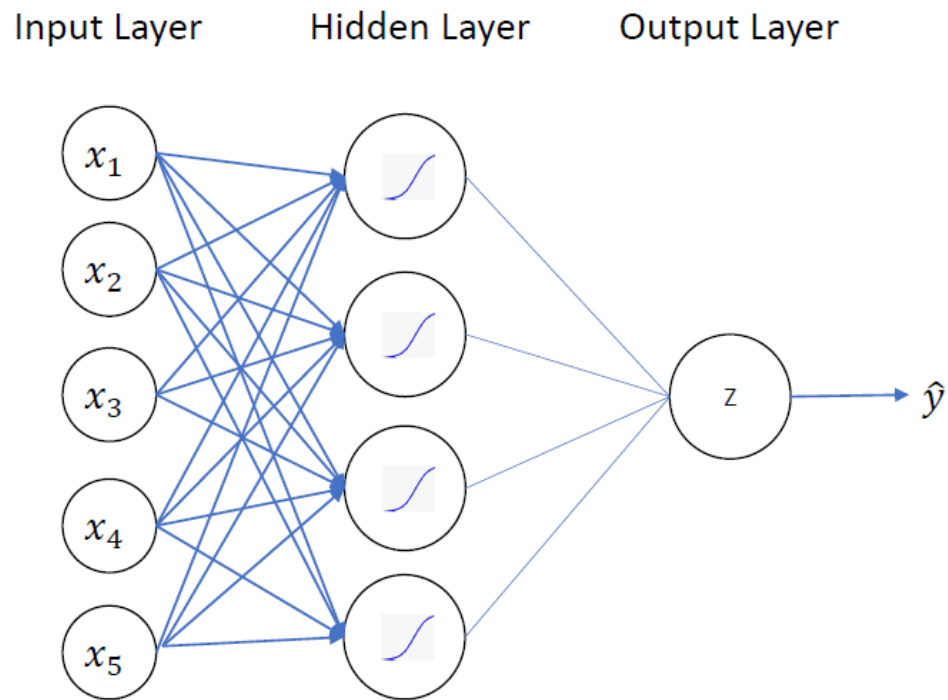


$$\text{Softmax } s(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

$$E(w) = -\frac{1}{m} \sum_i^m \sum_j^N y_{ij} \log \hat{y}_{ij}$$

- m is the number of rows
- N is the number of classes

Neural Network for Regression Problem



Mean Squared Error

$$E(W) = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)$$

References

<https://classroom.udacity.com/courses/ud188>

You tube Channels:



Books:

